HANDS-ON BUILD GUIDE · V2 · 30 OCT 2026 · LEONARD DUMITRU

3 AI projects, step by step

RAG over your own docs · Long-horizon agents · Deploy & load-test a model.
From zero setup to a GitHub repo with measured results.

01
RAG with citations
Chroma + bge-small + OpenRouter. Numbered sources, "I don't know" guard, eval with a pass rate.

02
Agents that act
Pydantic AI + typed tools + E2B sandbox. Parallel sub-agents, budget caps, human approval, JSON logs.

03
Deploy a model
vLLM on a free Colab T4. Measure TTFT, inter-token latency, throughput under load.

How to use this guide. Do Part 0 once (pick *No-IDE* or *IDE* track), then each project in order. Every command block says where it runs. Grey panels are files: create them with exactly that name and paste the content. The code was run on 30 Oct 2026 on a CPU-only Linux machine (RAG on CPU end-to-end except the LLM call; agents offline with a test model; the load-test scripts against a fake server). Anything not run is flagged `unverified`.

Leonard Dumitru · Inspired by the carousel "3 AI projects I'd build if I started again" (@khaliladib11).

Contents

Map	End-to-end journey map: setup → 01 → 02 → 03 → publish, with done-when gates
Part 0	Setup: No-IDE (Colab) track · IDE track (VS Code / Cursor, Python 3.11, venv, git, .env, API keys, Docker)
Project 01	RAG: detailed flow + 12 checkpoints · build steps · worked evals (real retrieval rows, judge card) · reviewer's checklist · error gallery
Project 02	Agents: flow + swimlane sequence + checkpoints · build steps · JSONL timeline & test-suite table · reviewer's checklist · error gallery
Project 03	Deploy: flow + checkpoints · build steps · metrics table & latency-vs-concurrency chart · reviewer's checklist · error gallery
Diagnose	Decision trees: answer is wrong · agent failed · endpoint slow
Read	How to read an eval report — annotated example and five traps
Part 4	Publish on GitHub: README what / how / results
Appendix	Versions tested, what was not verified, links

Total time

Part	Realistic time	Money
Part 0 setup	30 min (Colab) – 90 min (IDE + Docker)	€0
01 RAG	4–8 h (a weekend day)	< $1 of OpenRouter credit
02 Agents	6–10 h	< $2 OpenRouter; E2B gives new accounts free credit
03 Deploy	2–4 h	€0 on free Colab T4 (session limits apply)

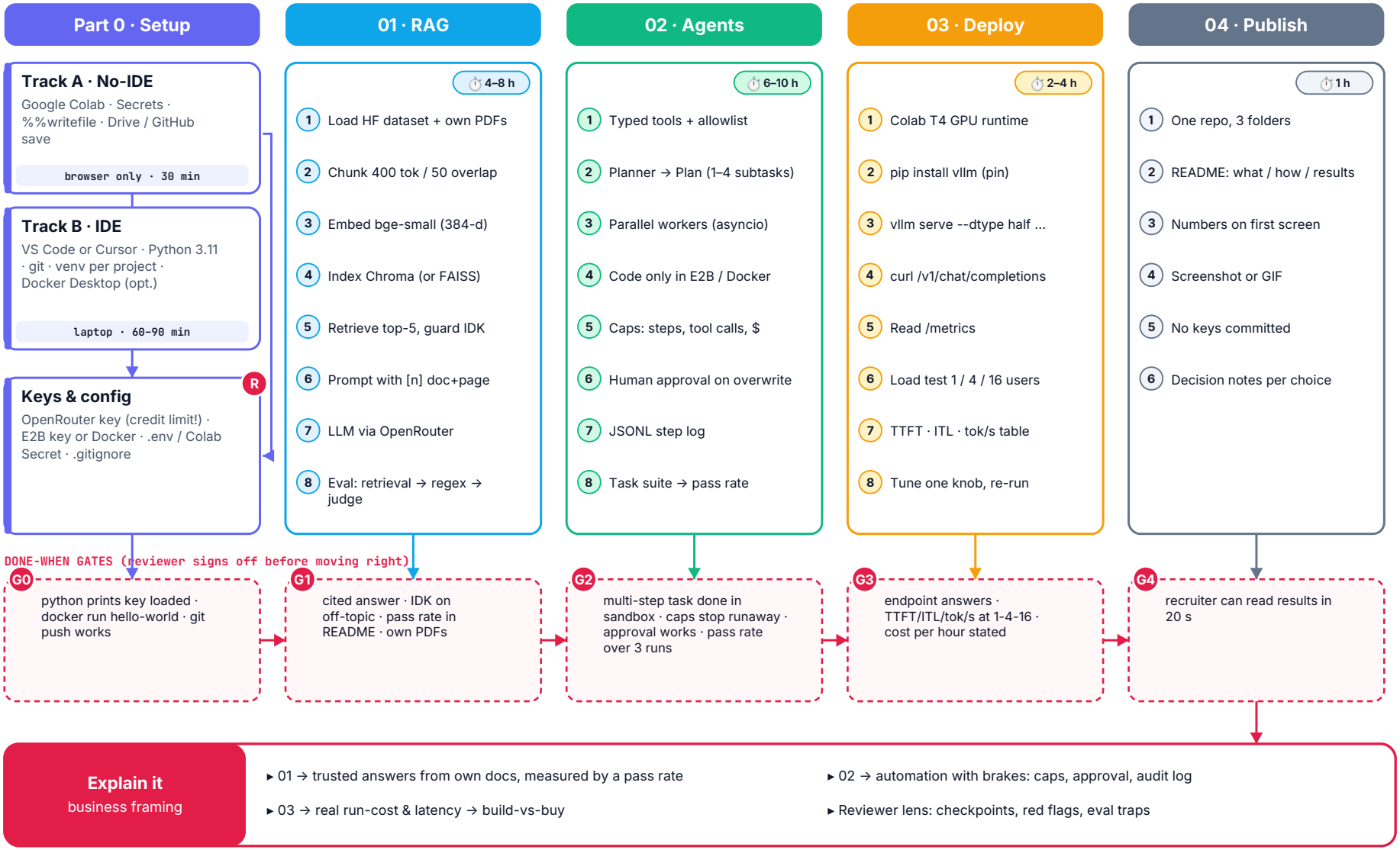
Time estimates are my own, for someone comfortable with a terminal but new to these libraries — not from the carousel.

Reading the visuals

3 red numbered badges on diagrams = review checkpoints, explained in the table after each flow. `measured` = numbers from an actual run on 30 Oct 2026 (CPU only). `illustrative values` = realistic examples to show the format; replace with your own measurements.

Map End-to-end journey map

From an empty laptop to three measured projects. Columns are phases; numbered dots are build steps; dashed red boxes are the **done-when gates** a reviewer signs off before work moves right. Use it as the plan for a build and as the review route for someone else's.



Part 0 Setup

You need three things whatever track you choose: **a place to run Python**, **an LLM API key** (OpenRouter: one key, hundreds of models, OpenAI-compatible API), and for Project 02 **a sandbox** (E2B key, or Docker Desktop locally).

	No-IDE track: Google Colab	IDE track: VS Code / Cursor
Install	Nothing. A browser and a Google account.	Python 3.11, git, editor, (Docker)
GPU	Free T4 (needed for Project 03)	None needed for 01/02. 03 still uses Colab or Lightning AI
Best for	Trying it today, demoing quickly	A real GitHub repo, the "production" story
Downside	Files vanish when the session ends; save to Drive/ GitHub	30–90 min setup

Recommendation: do Projects 01 and 02 in the IDE track (that is what makes the GitHub repo credible) and Project 03 in Colab (free GPU). If you only have an hour, do everything in Colab first.

Step 0.1 — Get your API keys (both tracks)

1 OpenRouter key (LLM calls)

1. Go to **openrouter.ai** → Sign in (Google/GitHub).
2. Top-right menu → **Credits** → add $5 (enough for all three projects many times over). Some models are free (`:free` suffix) but are rate-limited.
3. Open **openrouter.ai/keys** → **Create Key** → name it `ai-projects` → optionally set a credit limit (e.g. $3) → copy the key (starts `sk-or-v1-`). You will not see it again.
4. Pick a model id from **openrouter.ai/models**. This guide uses `openai/gpt-4o-mini` (cheap, good at tool calling). Any id on that page works.

2 E2B key (sandbox for Project 02)

1. Go to **e2b.dev** → Sign up → Dashboard → **API Keys** tab → copy the key (starts `e2b_`).
2. New accounts get free usage credit (E2B's docs say $100 at the time of writing) — no card needed to start.
3. Prefer local? Skip E2B and use Docker Desktop (Step 0.8); the code supports both.

Never paste keys into code or commit them. Keys live in a `.env` file (IDE track) or Colab Secrets (Colab track). `.env` is in `.gitignore` in every project folder below.

Track A — No-IDE: Google Colab

A1 Create a notebook

colab.research.google.com → **New notebook**. Rename it (top-left) to `01-rag.ipynb`.

A2 Store your keys as Colab Secrets

Left sidebar → 🔑 **Secrets** → **Add new secret** → name `OPENROUTER_API_KEY`, paste value, toggle **Notebook access** on. Repeat for `E2B_API_KEY`. Then in the first cell:

Colab cell 1

```
import os
from google.colab import userdata
os.environ["OPENROUTER_API_KEY"] = userdata.get("OPENROUTER_API_KEY")
# os.environ["E2B_API_KEY"] = userdata.get("E2B_API_KEY")   # Project 02
```

A3 Create the project files inside Colab

Each file in this guide becomes one cell that starts with `%%writefile filename`, followed by the file content. Run the cell and the file appears in the 📁 Files panel. Then run scripts with `!python script.py`:

```
Colab cell
%%writefile config.py
# ...paste the content of config.py from Project 01 here...
```

```
Colab cell
!pip install -q sentence-transformers chromadb datasets pypdf openai python-dotenv
!python ingest.py --hf --reset
!python rag.py "Was Abraham Lincoln the sixteenth President of the United States?"
```

A4 Do not lose your work

Colab deletes files when the runtime stops. **File → Save a copy in GitHub** for the notebook, or mount Drive: `from google.colab import drive; drive.mount('/content/drive')` and `%cd /content/drive/MyDrive/ai-projects` before writing files.

A5 GPU (only Project 03)

Runtime → Change runtime type → Hardware accelerator: T4 GPU → Save. Check with `!nvidia-smi`.

Track B — IDE: VS Code or Cursor on your laptop

B1 Install Python 3.11, git and an editor

Either download Python 3.11 from **python.org/downloads** (Windows: tick **"Add python.exe to PATH"** on the first screen), or use a package manager:

```
macOS (Terminal)
# Homebrew first, if you don't have it: https://brew.sh
brew install python@3.11 git
brew install --cask visual-studio-code   # or: brew install --cask cursor
python3.11 --version   # -> Python 3.11.x
git --version
```

```
Windows PowerShell
winget install -e --id Python.Python.3.11
winget install -e --id Git.Git
winget install -e --id Microsoft.VisualStudioCode
# Cursor: download the installer from the Cursor website
# close + reopen PowerShell, then:
py -3.11 --version     # -> Python 3.11.x
git --version
```

> **VS Code or Cursor?** Same editor underneath (Cursor is a VS Code fork with built-in AI). Install the **Python** extension (Microsoft) in either. Python 3.11 is a safe choice; the code was also run on 3.13.

B2 Configure git once

```
Terminal
git config --global user.name "Leonard Dumitru"
git config --global user.email "you@example.com"   # same email as your GitHub account
```

B3 Create the repo folder layout

One repo, one folder per project — recruiters can open one link and see all three:

Folder layout

```
ai-projects/
├── README.md                # 3 lines per project + links (Part 4)
├── 01-rag/
│   ├── .env  .env.example  .gitignore  requirements.txt
│   ├── config.py  ingest.py  rag.py  eval.py  faiss_store.py
│   ├── my_docs/             # your PDFs go here
│   └── chroma_db/           # created by ingest.py (git-ignored)
├── 02-agents/
│   ├── .env  .env.example  .gitignore  requirements.txt
│   ├── agent.py  sandbox.py  logger.py  run_tests.py
│   ├── tests/tasks.json
│   ├── workspace/           # the only folder the agent may touch
│   └── logs/                # JSONL step logs (git-ignored)
└── 03-deploy/
    ├── colab_cells.py  load_test.py  metrics_summary.py  fake_server.py
    ├── requirements.txt
    └── README.md            # results table
```

macOS (Terminal)

```
mkdir -p ~/code/ai-projects && cd ~/code/ai-
projects
mkdir -p 01-rag/my_docs 02-agents/tests 02-agents/
workspace 03-deploy
git init
code .        # or: cursor .
```

Windows PowerShell

```
mkdir $HOME\code\ai-projects; cd $HOME\code\ai-
projects
mkdir 01-rag\my_docs, 02-agents\tests, 02-agents\w
orkspace, 03-deploy
git init
code .        # or: cursor .
```

B4 One virtual environment per project

A venv keeps each project's packages separate. Do this inside each project folder (shown for 01-rag):

macOS (Terminal)

```
cd 01-rag
python3.11 -m venv .venv
source .venv/bin/activate     # prompt now starts
with (.venv)
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Windows PowerShell

```
cd 01-rag
py -3.11 -m venv .venv
# if activation is blocked, run once:
#   Set-ExecutionPolicy -Scope CurrentUser
RemoteSigned
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

In VS Code/Cursor: **Ctrl/Cmd+Shift+P → "Python: Select Interpreter"** → pick `.venv`. New terminals then activate it automatically. Run `deactivate` to leave.

B5 The .env file and .gitignore

Copy `.env.example` to `.env` and paste your real keys. `python-dotenv` loads it at start-up.

macOS (Terminal)

```
cp .env.example .env
```

Windows PowerShell

```
Copy-Item .env.example .env
```

.gitignore

```
.env
.venv/
chroma_db/
__pycache__/
```

B6 Docker Desktop (optional — local sandbox for Project 02)

macOS (Terminal)

```
brew install --cask docker
open -a Docker        # wait for the whale icon
docker run --rm hello-world
```

Windows PowerShell

```
winget install -e --id Docker.DockerDesktop
# Docker Desktop uses WSL 2; accept the prompt
# to install/enable it, then reboot
docker run --rm hello-world
```

Then pre-pull the image the agent will use: `docker pull python:3.11-slim`.

B7 First push to GitHub

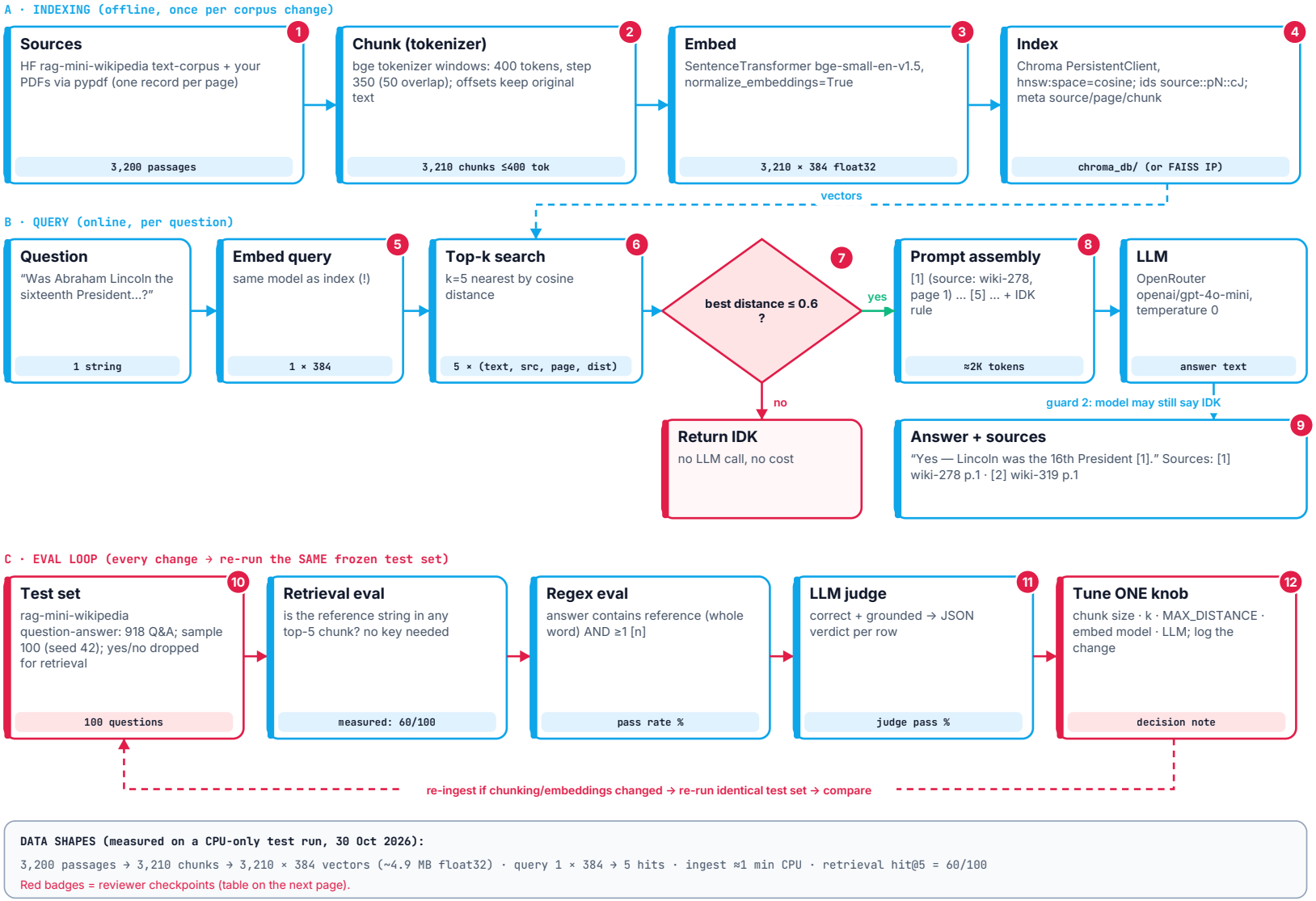
Terminal

```
# create an EMPTY repo called ai-projects on github.com first (no README), then:
git add . && git commit -m "Scaffold three AI projects"
git branch -M main
git remote add origin https://github.com/<you>/ai-projects.git
git push -u origin main
```

Check on github.com that no `.env` file was pushed.

01 · Flow RAG — detailed flow with checkpoints

Three loops: indexing (offline), query (online), eval (every change). Red badges are the places to look when reviewing someone's RAG system.



DATA SHAPES (measured on a CPU-only test run, 30 Oct 2026):

3,200 passages → 3,210 chunks → 3,210 × 384 vectors (~4.9 MB float32) · query 1 × 384 → 5 hits · ingest ≈1 min CPU · retrieval hit@5 = 60/100

Red badges = reviewer checkpoints (table on the next page).

01 · Checkpoints

RAG — checkpoints

Checkpoint guide — Project 01

#	Checkpoint — what to inspect	Good looks like	Red flags
1	Sources: which docs, how many, how extracted	Inventory with counts; PDFs extracted as text per page; scanned PDFs OCR'd	"We indexed SharePoint" with no count; empty pages; tables flattened into noise
2	Chunking: size, overlap, unit	Size in *model tokens* with rationale (300–500, ~50 overlap); 5 sample chunks shown	Sizes in characters only; chunks larger than the embed model's limit (MiniLM 256); cut mid-sentence
3	Embedding model and limits	Named model + version; max tokens ≥ chunk size; same model at query time	Model swapped without re-indexing; vectors not normalised but cosine assumed
4	Index and metadata	Every chunk has source + page + chunk id; count = expected	Missing page metadata → citations impossible; duplicates from re-ingesting
5	Query embedding	Exactly the index model	Different model / dimension → silently bad results
6	Top-k results	Hit rate on a labelled set; distances visible; k justified	No retrieval metric at all; near-duplicate chunks fill the top-k
7	IDK guard / threshold	Threshold chosen from data; IDK rate reported on in-scope AND off-topic questions	Never says IDK (hallucinates) or says IDK to 30% of answerable questions
8	Prompt assembly	Numbered passages with source+page; rules to cite and refuse; token budget	Unnumbered context; citations generated from memory; 20 chunks stuffed in
9	Answer and citations	Every claim has [n]; [n] ≤ k; cited chunk actually supports it	Citations to [7] when k=5; page numbers not in metadata
10	Eval set	≥100 questions, how built, frozen, separate from tuning examples	20 hand-picked questions; same questions used to tune and to report
11	LLM judge	Rubric written down; judge model ≠ answer model; spot-checked vs humans	Judge = same model grading itself; no human agreement check
12	Tuning log	One change at a time, before/after numbers, decision recorded	Many knobs changed at once; only the best run reported

Project 01 RAG over your own documents

⏱️ 4–8 hours | 💶 < $1 LLM credit; embeddings free on CPU | 🧰 IDE or Colab, no GPU

Goal. Ask a question about a document collection and get an answer that cites numbered sources — document and page — and says *"I don't know"* when the documents don't contain the answer. Then prove it works with an evaluation that prints a pass rate.

PDFs / HF dataset → chunk 300–500 tokens, ~50 overlap → embed (bge-small) → Chroma → top-k retrieve → prompt with [1][2] →

Decisions (and why)

Choice	Default here	Alternatives from the carousel
Embeddings	`BAAI/bge-small-en-v1.5` (384-dim, runs on CPU, free)	`all-MiniLM-L6-v2` (faster; but it only reads the first 256 word-pieces, so it silently truncates 400-token chunks) · OpenAI `text-embedding-3-small` (paid API)
Vector store	Chroma (local folder, zero setup)	FAISS (library, in-memory, very fast) · pgvector (Postgres extension — the "production" answer)
Chunks	400 tokens, 50 overlap, counted with the embedding model's own tokenizer	300–500 is the carousel's range; smaller = precise, larger = more context
Context to LLM	top 5 chunks (≈2K tokens)	Carousel: top 3–5 chunks, ~2–3K tokens
LLM	`openai/gpt-4o-mini` through OpenRouter	any model id from openrouter.ai/models

Step-by-step

1 Create the folder, requirements and venv

In `ai-projects/01-rag/` create:

requirements.txt

```
# Project 01 - RAG over your own docs (tested on Python 3.11-3.13)
sentence-transformers>=3.0
chromadb>=1.0
datasets>=3.0
pypdf>=5.0
openai>=1.40
python-dotenv>=1.0
faiss-cpu>=1.8      # optional: only for faiss_store.py
```

.env.example

```
OPENROUTER_API_KEY=sk-or-v1-paste-your-key-here
# optional overrides
# LLM_MODEL=openai/gpt-4o-mini
# EMBED_MODEL=sentence-transformers/all-MiniLM-L6-v2
```

Then create and activate the venv and install (Part 0, B4). First install downloads PyTorch (~1–2 GB on Windows/macOS) — be patient.

2 config.py — all settings in one place

config.py

```python
"""Shared settings for the RAG project. Change values here, not in every file."""
import os
from dotenv import load_dotenv

load_dotenv()  # reads .env in the current folder (OPENROUTER_API_KEY=...)

# all-MiniLM-L6-v2 only embeds the first 256 word-pieces of a chunk.
# bge-small-en-v1.5 embeds up to 512, so it fits 300-500 token chunks.
EMBED_MODEL = os.getenv("EMBED_MODEL", "BAAI/bge-small-en-v1.5")
CHUNK_TOKENS = int(os.getenv("CHUNK_TOKENS", "400"))
CHUNK_OVERLAP = int(os.getenv("CHUNK_OVERLAP", "50"))
TOP_K = int(os.getenv("TOP_K", "5"))
# Cosine distance in Chroma = 1 - cosine similarity. Above this = "not relevant".
MAX_DISTANCE = float(os.getenv("MAX_DISTANCE", "0.6"))

DB_DIR = os.getenv("DB_DIR", "chroma_db")
COLLECTION = os.getenv("COLLECTION", "docs")

LLM_MODEL = os.getenv("LLM_MODEL", "openai/gpt-4o-mini")  # any id from openrouter.ai/models
OPENROUTER_BASE_URL = "https://openrouter.ai/api/v1"
```

3 ingest.py — load, chunk, embed, store

Reads the Hugging Face dataset `rag-datasets/rag-mini-wikipedia` (3,200 Wikipedia passages + 918 Q&A pairs, config `text-corpus`, split `passages`) and/or your own PDFs page by page with `pypdf`, so every chunk remembers **source and page**.

ingest.py

```python
"""Step 1-4: load -> chunk -> embed -> index (Chroma, persisted to disk).

Usage:
  python ingest.py --hf --limit 3200      # rag-mini-wikipedia passages
  python ingest.py --pdfs my_docs/        # your own PDFs (one record per page)
  python ingest.py --hf --pdfs my_docs/   # both
"""
import argparse
import pathlib

import chromadb
from sentence_transformers import SentenceTransformer

import config


def load_hf(limit: int | None):
    from datasets import load_dataset
    ds = load_dataset("rag-datasets/rag-mini-wikipedia", "text-corpus", split="passages")
    if limit:
        ds = ds.select(range(min(limit, len(ds))))
    for row in ds:
        # each passage is one "page" of a pseudo-document called wiki-<id>
        yield {"text": row["passage"], "source": f"wiki-{row['id']}", "page": 1}


def load_pdfs(folder: str):
    from pypdf import PdfReader
    for pdf in sorted(pathlib.Path(folder).glob("**/*.pdf")):
        reader = PdfReader(str(pdf))
        for i, page in enumerate(reader.pages, start=1):
            text = page.extract_text() or ""
            if text.strip():
                yield {"text": text, "source": pdf.name, "page": i}


def chunk(text: str, tokenizer, size: int, overlap: int) -> list[str]:
    """Token-based sliding window so 'size' really means model tokens.
    We slice the ORIGINAL text via character offsets, so case and spacing survive."""
    enc = tokenizer(text, add_special_tokens=False, return_offsets_mapping=True)
    offsets = enc["offset_mapping"]
    if not offsets:
        return []
    step = max(1, size - overlap)
    out = []
    for start in range(0, len(offsets), step):
        window = offsets[start:start + size]
        out.append(text[window[0][0]:window[-1][1]].strip())
        if start + size >= len(offsets):
            break
    return out


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--hf", action="store_true", help="index rag-mini-wikipedia")
    ap.add_argument("--limit", type=int, default=None, help="max HF passages")
    ap.add_argument("--pdfs", default=None, help="folder with your PDFs")
    ap.add_argument("--reset", action="store_true", help="drop the collection first")
    args = ap.parse_args()
    if not (args.hf or args.pdfs):
        ap.error("pass --hf and/or --pdfs FOLDER")

    model = SentenceTransformer(config.EMBED_MODEL)
```

```
    tok = model.tokenizer

    client = chromadb.PersistentClient(path=config.DB_DIR)
    if args.reset:
        try:
            client.delete_collection(config.COLLECTION)
        except Exception:
            pass
    col = client.get_or_create_collection(config.COLLECTION, metadata={"hnsw:space": "cosine"})

    records = []
    if args.hf:
        records += list(load_hf(args.limit))
    if args.pdfs:
        records += list(load_pdfs(args.pdfs))
    print(f"loaded {len(records)} pages/passages")

    ids, docs, metas = [], [], []
    for r in records:
        for j, c in enumerate(chunk(r["text"], tok, config.CHUNK_TOKENS, config.CHUNK_OVERLAP)):
            ids.append(f"{r['source']}::p{r['page']}::c{j}")
            docs.append(c)
            metas.append({"source": r["source"], "page": r["page"], "chunk": j})
    print(f"made {len(docs)} chunks (size={config.CHUNK_TOKENS}, overlap={config.CHUNK_OVERLAP})")

    B = 256
    for i in range(0, len(docs), B):
        vecs = model.encode(docs[i:i + B], normalize_embeddings=True, show_progress_bar=False)
        col.upsert(ids=ids[i:i + B], documents=docs[i:i + B],
                   metadatas=metas[i:i + B], embeddings=vecs.tolist())
        print(f"  indexed {min(i + B, len(docs))}/{len(docs)}")
    print(f"done. collection '{config.COLLECTION}' has {col.count()} chunks in ./{config.DB_DIR}")


if __name__ == "__main__":
    main()
```

4 Run ingestion

Terminal

```
python ingest.py --hf --limit 300 --reset   # quick test: 300 passages
python ingest.py --hf --reset               # full dataset (~1 min on a modern CPU)
# your own PDFs: drop them into my_docs/ then
python ingest.py --pdfs my_docs
```

Expected: `done. collection 'docs' has 3210 chunks in ./chroma_db` (3,200 passages → 3,210 chunks: a few long passages are split). The first run downloads the 130 MB embedding model.

5 rag.py — retrieve, build the cited prompt, answer

Two “I don't know” guards: **(1)** if even the best chunk is too far away (cosine distance > 0.6), we return IDK *without calling the LLM* (cheaper, no hallucination); **(2)** the system prompt orders the model to answer only from the passages and reply IDK otherwise.

rag.py

```python
"""Step 5-6: retrieve top-k chunks and answer with numbered citations.

Usage:
  python rag.py "Who was the sixteenth president of the United States?"
  python rag.py --no-llm "question"     # retrieval only, no API key needed
"""
import argparse
import os
from functools import lru_cache

import chromadb
from openai import OpenAI
from sentence_transformers import SentenceTransformer

import config

IDK = "I don't know based on the provided documents."

SYSTEM = (
    "You answer questions using ONLY the numbered context passages. "
    "After every sentence that uses a passage, cite it like [1] or [2][3]. "
    f"If the passages do not contain the answer, reply exactly: {IDK} "
    "Never use outside knowledge. Be concise."
)


@lru_cache
def _model():
    return SentenceTransformer(config.EMBED_MODEL)


@lru_cache
def _collection():
    client = chromadb.PersistentClient(path=config.DB_DIR)
    return client.get_collection(config.COLLECTION)


@lru_cache
def _llm():
    key = os.getenv("OPENROUTER_API_KEY")
    if not key:
        raise SystemExit("OPENROUTER_API_KEY missing: put it in .env")
    return OpenAI(base_url=config.OPENROUTER_BASE_URL, api_key=key)


def retrieve(question: str, k: int = config.TOP_K) -> list[dict]:
    q = _model().encode([question], normalize_embeddings=True).tolist()
    res = _collection().query(query_embeddings=q, n_results=k,
                              include=["documents", "metadatas", "distances"])
    hits = []
    for doc, meta, dist in zip(res["documents"][0], res["metadatas"][0], res["distances"][0]):
        hits.append({"text": doc, "source": meta["source"], "page": meta["page"], "distance": dist})
    return hits


def build_prompt(question: str, hits: list[dict]) -> str:
    blocks = [f"[{i}] (source: {h['source']}, page {h['page']})\n{h['text']}"
              for i, h in enumerate(hits, start=1)]
    return "Context passages:\n\n" + "\n\n".join(blocks) + f"\n\nQuestion: {question}\nAnswer:"


def answer(question: str, k: int = config.TOP_K, use_llm: bool = True) -> dict:
    hits = [h for h in retrieve(question, k) if h["distance"] <= config.MAX_DISTANCE]
    if not hits:  # guard 1: nothing relevant retrieved -> refuse without calling the LLM
```

```
        return {"answer": IDK, "sources": []}
    if not use_llm:
        return {"answer": "(LLM skipped)", "sources": hits}
    resp = _llm().chat.completions.create(
        model=config.LLM_MODEL,
        temperature=0,
        messages=[{"role": "system", "content": SYSTEM},
                  {"role": "user", "content": build_prompt(question, hits)}],
    )
    text = resp.choices[0].message.content.strip()  # guard 2: the system prompt
    return {"answer": text, "sources": hits, "usage": resp.usage}


def format_sources(hits: list[dict]) -> str:
    return "\n".join(f"  [{i}] {h['source']} p.{h['page']}  (distance {h['distance']:.3f})"
                     for i, h in enumerate(hits, start=1))


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("question")
    ap.add_argument("--no-llm", action="store_true")
    a = ap.parse_args()
    out = answer(a.question, use_llm=not a.no_llm)
    print(out["answer"])
    if out["sources"]:
        print("\nSources:\n" + format_sources(out["sources"]))
```

6 Ask questions

```
Terminal
python rag.py --no-llm "Was Abraham Lincoln the sixteenth President of the United States?"   # retrieval only
python rag.py "Was Abraham Lincoln the sixteenth President of the United States?"
python rag.py "What is the capital of Mars?"     # should say: I don't know...
```

Real output of the retrieval-only run (CPU, 30 Oct 2026):

```
output
Sources:
  [1] wiki-278 p.1  (distance 0.122)
  [2] wiki-319 p.1  (distance 0.226)
  [3] wiki-288 p.1  (distance 0.274)
  [4] wiki-1992 p.1  (distance 0.313)
  [5] wiki-198 p.1  (distance 0.326)
```

7 eval.py — measure it, print a pass rate

Three levels: `--retrieval` (no key: does the reference answer appear in the top-k chunks?), default (regex: answer contains the reference and at least one [n] citation), and `--judge` (an LLM grades *correct* and *grounded*, returns JSON).

eval.py

```python
"""Step 7: evaluation. Regex first (cheap, deterministic), LLM judge second.

Usage:
  python eval.py --n 50              # regex eval on 50 test questions
  python eval.py --n 50 --judge      # add an LLM-as-judge score
  python eval.py --n 50 --retrieval  # retrieval hit-rate only, no API key needed
"""
import argparse
import json
import random
import re

from datasets import load_dataset

import config
import rag

JUDGE_PROMPT = """You grade a RAG answer. Reply with JSON only:
{{"correct": true|false, "grounded": true|false, "reason": "..."}}
correct  = the answer agrees with the reference answer.
grounded = every claim is supported by the cited passages.

Question: {q}
Reference answer: {ref}
Candidate answer: {a}
Cited passages:
{ctx}"""


def load_tests(n: int, seed: int = 42, skip_yes_no: bool = False):
    ds = load_dataset("rag-datasets/rag-mini-wikipedia", "question-answer", split="test")
    rows = list(ds)
    if skip_yes_no:  # "yes"/"no" never appears verbatim in a passage
        rows = [r for r in rows if r["answer"].strip().lower().rstrip(".") not in ("yes", "no")]
    random.Random(seed).shuffle(rows)
    return rows[:n]


def regex_for(expected: str) -> str:
    # whole-word, case-insensitive match of the reference answer
    return r"\b" + re.escape(expected.strip().rstrip(".")) + r"\b"


def judge(q, ref, a, hits) -> dict:
    ctx = "\n".join(f"[{i}] {h['text'][:800]}" for i, h in enumerate(hits, 1))
    resp = rag._llm().chat.completions.create(
        model=config.LLM_MODEL, temperature=0,
        response_format={"type": "json_object"},
        messages=[{"role": "user", "content": JUDGE_PROMPT.format(q=q, ref=ref, a=a, ctx=ctx)}],
    )
    try:
        return json.loads(resp.choices[0].message.content)
    except json.JSONDecodeError:
        return {"correct": False, "grounded": False, "reason": "judge returned bad JSON"}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--n", type=int, default=50)
    ap.add_argument("--judge", action="store_true")
    ap.add_argument("--retrieval", action="store_true",
                    help="only check whether the answer text appears in the top-k chunks")
    args = ap.parse_args()
```

```
    tests = load_tests(args.n, skip_yes_no=args.retrieval)
    passed = judged_ok = 0
    for t in tests:
        q, ref = t["question"], t["answer"]
        if args.retrieval:
            hits = rag.retrieve(q)
            ok = any(re.search(regex_for(ref), h["text"], re.I) for h in hits)
            passed += ok
            continue
        out = rag.answer(q)
        a = out["answer"]
        cited = re.search(r"\[\d+\]", a)
        correct = re.search(regex_for(ref), a, re.I)
        ok = bool(cited and correct)
        passed += ok
        line = f"{'PASS' if ok else 'FAIL'} | {q[:60]:60} | ref={ref[:20]!r}"
        if args.judge and out["sources"]:
            j = judge(q, ref, a, out["sources"])
            judged_ok += bool(j.get("correct") and j.get("grounded"))
            line += f" | judge={j.get('correct')}/{j.get('grounded')}"
        print(line)

    label = "retrieval hit rate" if args.retrieval else "pass rate"
    print(f"\n{label}: {passed / len(tests):.0%}  ({passed}/{len(tests)})")
    if args.judge:
        print(f"judge pass rate (correct AND grounded): {judged_ok / len(tests):.0%}")


if __name__ == "__main__":
    main()
```

Terminal

```
python eval.py --n 100 --retrieval   # measured: "retrieval hit rate: 60%  (60/100)"
python eval.py --n 50                # regex pass rate (needs key)
python eval.py --n 50 --judge        # + LLM-as-judge (needs key)
```

Read the number carefully. The 60% hit rate is a strict string match of the reference answer inside the chunks, with yes/no questions excluded (“yes” never appears verbatim in a passage). Many “misses” are paraphrases. That's exactly why you add the LLM judge.

8 faiss_store.py — the same search with FAISS

Shows the store is swappable: reads the vectors back from Chroma into an in-memory FAISS `IndexFlatIP` (inner product on normalised vectors = cosine similarity).

faiss_store.py

```python
"""Alternative index: FAISS in memory (no server, very fast for experiments).
Reads the chunks already stored in Chroma, so run ingest.py first.

  python faiss_store.py "Was Abraham Lincoln the sixteenth President?"
"""
import sys

import chromadb
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

import config

col = chromadb.PersistentClient(path=config.DB_DIR).get_collection(config.COLLECTION)
data = col.get(include=["documents", "metadatas", "embeddings"])
vecs = np.asarray(data["embeddings"], dtype="float32")   # already L2-normalised

index = faiss.IndexFlatIP(vecs.shape[1])   # inner product = cosine on normalised vectors
index.add(vecs)

model = SentenceTransformer(config.EMBED_MODEL)
q = model.encode([sys.argv[1]], normalize_embeddings=True).astype("float32")
scores, idx = index.search(q, config.TOP_K)
for rank, (s, i) in enumerate(zip(scores[0], idx[0]), start=1):
    m = data["metadatas"][i]
    print(f"[{rank}] {m['source']} p.{m['page']}  cos={s:.3f}  {data['documents'][i][:90]!r}")
```

Terminal

```
python faiss_store.py "Who was Abraham Lincoln's wife?"
```

9 Upgrade path: pgvector

For production, the same vectors go into Postgres with the `pgvector` extension: one database for data, permissions, backups and vectors (`CREATE EXTENSION vector;` then a `vector(384)` column and an HNSW index). Neon and Supabase offer it free to start. You don't need to build this — know when you'd move: many users, access control per document, or data already in Postgres.

Done when

- ☐ You can ask a question and the answer cites numbered sources with document + page
- ☐ An off-topic question returns “I don't know based on the provided documents.”
- ☐ `eval.py` prints a pass rate and you've written it in the README (and tried one change — chunk size or k — and recorded the effect)
- ☐ Your own PDFs work, not just the demo dataset
- ☐ Repo folder has README with what / how / results; no keys committed

Common errors & fixes

Symptom	Fix
`ModuleNotFoundError: chromadb`	venv not active, or VS Code uses another interpreter → activate `.venv`, “Python: Select Interpreter”.
`Collection [docs] does not exist`	Run `ingest.py` first, from the `01-rag` folder (DB_DIR is relative).

Torch install fails on Windows / very slow	Use Python 3.11 64-bit; or install CPU torch first: `pip install torch --index-url https://download.pytorch.org/whl/cpu`.
Answers are always “I don't know”	MAX_DISTANCE too strict for your docs → print the distances (`--no-llm`) and raise to 0.7; check PDFs actually have text (scanned PDFs need OCR).
`401` / `No auth credentials` from OpenRouter	`.env` not in the folder you run from, or key typo. `python -c "import config,os;print(bool(os.getenv('OPENROUTER_API_KEY')))"`
`402` from OpenRouter	No credits → add credit or use a `:free` model id.
Odd characters like `â€“` in chunks	That's in the dataset itself (encoding artefact); harmless for retrieval. `pip install ftfy` and `ftfy.fix_text()` if it bothers you.
HF warning “unauthenticated requests”	Harmless. Optional: create a free token at huggingface.co/settings/tokens and set `HF_TOKEN`.

01 · Evals RAG — evals, worked example

Questions, expected answers and retrieved chunk ids are **real** (bge-small, k=5, full 3,210-chunk index). Answer, citation and judge columns are `illustrative values` because they need an LLM call. Row 3 is the classic case: regex says fail, a human or judge says pass.

Question (real, rag-mini-wikipedia)	Expected	Retrieved top-3 (real)	Retrieval	Answer snippet `illustrative values`	Cited?	Correct?	Verdict
How many species of penguins are there?	between 17 and 20 living species	wiki-952 · wiki-953 · wiki-951	hit @1	“There are between 17 and 20 living species [1].”	✔	✔	PASS
When did Monroe's presidency expired?	March 4, 1825	wiki-2520 · wiki-2506 · wiki-2512	hit @1	“His presidency ended on March 4, 1825 [1].”	✔	✔	PASS
What are some economic uses for duck?	Meat,eggs,feathers	wiki-3181 · wiki-3164 · wiki-3180	miss (string)	“Ducks are farmed for meat, eggs and down feathers [1][2].”	✔	✔ (judge)	MIXED*
What did Ford receive on April 13, 1942?	Ford received a commission as ensign…	wiki-2012 · wiki-2013 · wiki-2019	miss (string)	“I don't know based on the provided documents.”	–	✘	FAIL

* MIXED = regex fail, judge pass: the reference “Meat,eggs,feathers” never appears verbatim. Question wording is copied as-is from the dataset, typos included.

Pass rates

Only the first bar is measured `measured`; the rest are `illustrative values` to show how a report should compare levels.



LLM-judge rubric card

Judge prompt (eval.py JUDGE_PROMPT)

```
You grade a RAG answer. Reply with JSON only:
{"correct": true|false, "grounded": true|false,
"reason": "..."}
correct  = the answer agrees with the reference
answer.
grounded = every claim is supported by the cited
passages.

Question: What are some economic uses for duck?
Reference answer: Meat,eggs,feathers
Candidate answer: Ducks are farmed for meat, eggs
  and down feathers [1][2].
Cited passages:
[1] ...  [2] ...
```

Verdict JSON — illustrative

```
{"correct": true,
 "grounded": true,
 "reason": "Mentions meat, eggs and feathers;
            passage [1] lists all three."}
```

Rubric

- **correct** — agrees with the reference (paraphrase OK)
- **grounded** — every claim traceable to a cited passage
- Row passes only if both are true
- Spot-check ~20 verdicts by hand; report judge-human agreement
- Use a different (ideally stronger) model than the one answering

01 · Review

RAG — reviewer's checklist

Questions to ask an engineer who built a RAG system, and the evidence to ask for. If they can't produce the artefact, treat the claim as unproven.

Ask the engineer	Artefact to demand	Good answer	Red flag
Why this chunk size and overlap?	Sample of 5 chunks + hit rate at 2–3 sizes	"400 tokens: 300 lost context, 500 dropped hit rate by x points"	"Default value", no experiment
Show me 5 random chunks.	Printed chunks with source/page	Readable, complete sentences, right metadata	Mid-sentence cuts, headers/ footers only, broken encoding
What's the retrieval hit rate on a labelled set?	eval --retrieval output, n and k	Number + n (e.g. 60/100 @k=5) + what a miss looks like	Only end-to-end demos
How accurate are citations?	Judge 'grounded' rate + 20 hand-checked rows	Grounded ≥ correct; disagreements explained	Never checked whether [n] supports the claim
What's the 'I don't know' rate?	IDK % on answerable and on off-topic questions	Low on answerable, high on off-topic	Zero IDKs (hallucination) or high IDK on answerable
How was the eval set built? How big?	File + build script + date frozen	≥100, mix of easy/hard, written before tuning	Picked after seeing results; < 30 questions
Any leakage between tuning and eval?	Separate dev and test splits	Tuned on dev, reported once on test	Same questions used to choose k and to report
Latency and cost per query?	Log of tokens in/out, ms per stage	p50/p95 latency, €/1K queries	"It's cheap" without numbers
What happens when documents change?	Re-ingest procedure, ids, deletion	Upsert by stable ids, stale chunks removed	Duplicates pile up; old versions still cited

01 · Errors

RAG — error gallery

Realistic failures, how to spot each one in the artefacts, and the usual root cause. Snippets are `illustrative values` (representative, not copied from a real system).

Citation that doesn't support the claim

```
Q: When did Monroe leave office?
A: Monroe left office in 1825 and
   retired to Virginia [3].
[3] …Monroe Doctrine was declared in 1823…
```

Symptom	Answer looks cited, but [3] says nothing about retiring.
How to spot	Judge 'grounded'=false; open the cited chunk in the eval row.
Likely causes	Model cites from memory; too many similar chunks; no per-claim quote rule.
Fix	Require a quote per claim; judge groundedness; fewer, better chunks.

Chunk cut mid-sentence

```
…Lincoln married Mary Todd, daughter of a
prominent slave-owning family from Lexi
——— chunk boundary ———
ngton, Kentucky. They had four sons…
```

Symptom	Answers miss facts that span chunk boundaries.
How to spot	Print random chunks; retrieval misses on boundary facts.
Likely causes	Fixed character windows, no overlap, decoded tokens instead of text offsets.
Fix	Token windows with overlap (~50), split on sentence boundaries, keep offsets.

Hallucinated page number

```
A: The policy allows 25 days [2].
Sources: [2] handbook.pdf p.47
# handbook.pdf has 31 pages
```

Symptom	Citation points to a page that doesn't exist.
How to spot	Compare cited page with chunk metadata / PDF page count.
Likely causes	Model writes the page itself instead of the system rendering sources.
Fix	Render sources from metadata in code, never let the model write them.

Near-duplicate chunks in top-k

```
[1] wiki-2520 dist 0.201
[2] wiki-2520-copy dist 0.201
[3] wiki-2520 (v2) dist 0.203
[4] …
```

Symptom	Top-5 holds 3 copies of the same text; other evidence crowded out.
How to spot	Identical distances / texts in --no-llm output; chunk count > expected.
Likely causes	Re-ingesting without reset; same doc in two folders; overlap too large.
Fix	Stable ids + upsert, dedupe by hash, MMR / diversity in retrieval.

Regex eval passes on a generic word

```
ref: "Three."   regex: \bThree\b
A: "I can't find all three suborders."
→ PASS
```

Symptom	Pass rate looks high but answers are wrong.
How to spot	Read 20 PASS rows by hand; short references (yes/no/numbers) dominate.
Likely causes	Exact-match on short answers; no citation or correctness check.
Fix	Exclude/flag short refs, add LLM judge, sample-check passes.

IDK guard firing on answerable questions

```
MAX_DISTANCE=0.25
Q: What did Ford receive on April 13, 1942?
best distance 0.280 → IDK
```

Symptom	Safe but useless: many 'I don't know' answers.
How to spot	IDK rate on answerable questions; distance distribution plot.
Likely causes	Threshold set by guess; different embed model changed the distance scale.
Fix	Pick the threshold from data (answerable vs off-topic distances), re-check after model change.

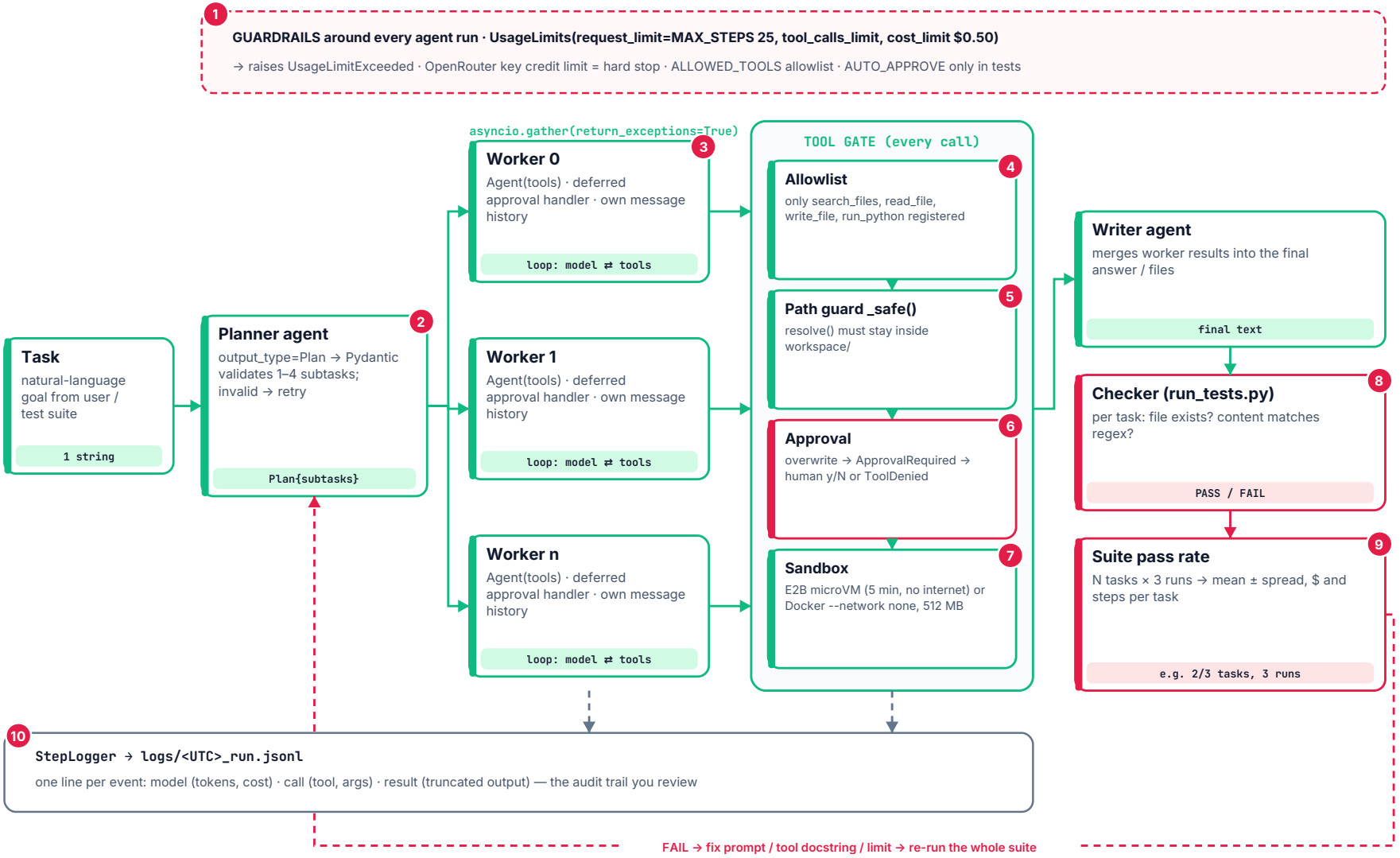
Eval set leakage

```
dev.json == test.json
# k, chunk size and prompt tuned
# on the same 50 questions reported
```

Symptom	Great reported score, disappointing in production.
How to spot	Ask how the test set was built and when it was frozen.
Likely causes	Tuning and reporting on the same questions.
Fix	Split dev/test; tune on dev; report test once; refresh periodically.

02 · Flow

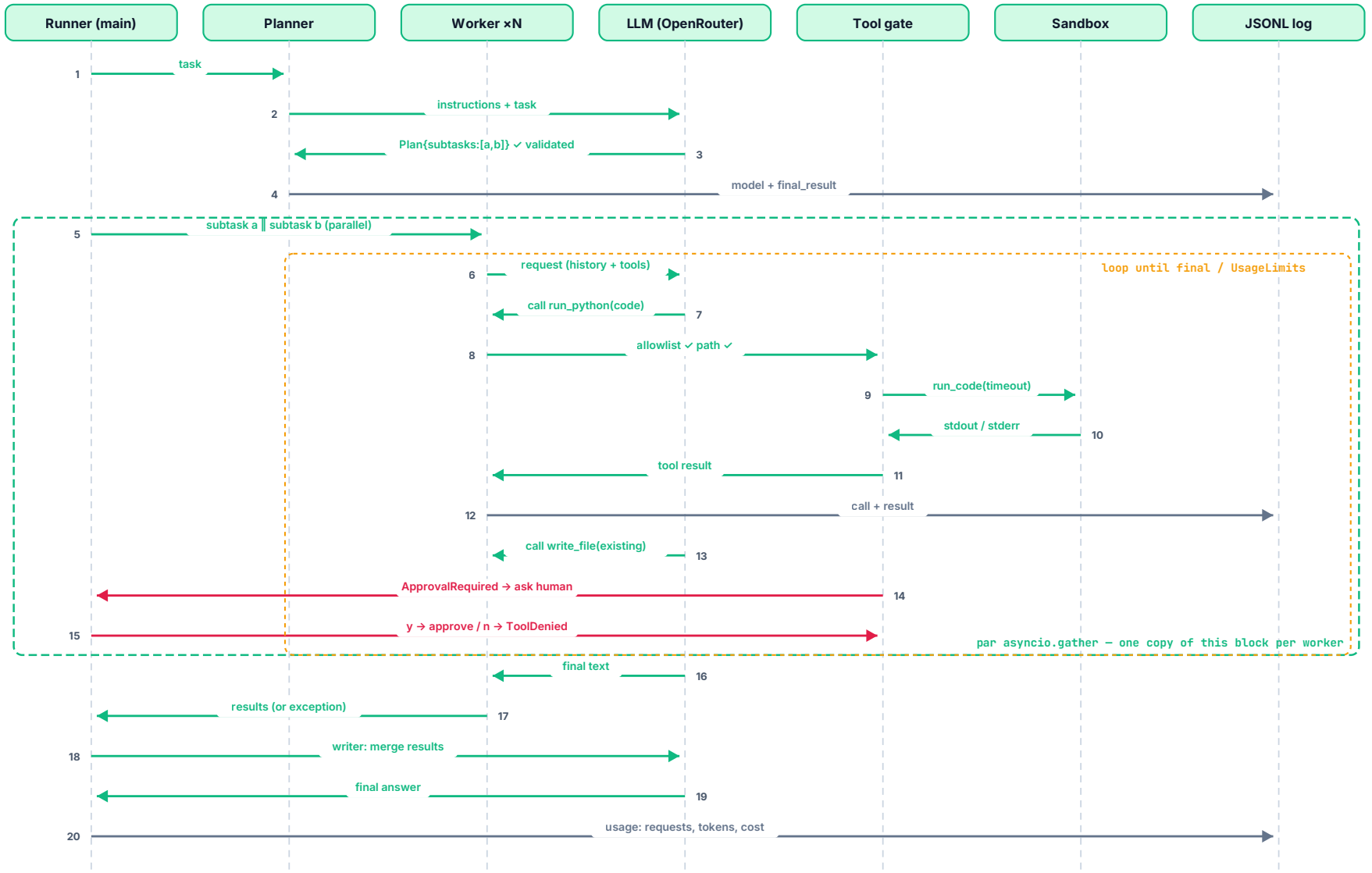
Agents — detailed flow with checkpoints



02 · Sequence

Agents — sequence (swimlanes)

One run, message by message: planner → parallel workers → tool gate → sandbox, with the approval round-trip in red and everything written to the JSONL log.



02 · Checkpoints

Agents — checkpoints

Checkpoint guide — Project 02

#	Checkpoint — what to inspect	Good looks like	Red flags
1	Guardrails: limits per run	request_limit, tool_calls_limit, cost cap set and tested (MAX_STEPS=2 stops)	No limits; or limits never triggered in tests
2	Planner output	Typed Plan validated; 1–4 sensible subtasks in the log	Free-text plans; 10 overlapping subtasks
3	Workers in parallel	gather with return_exceptions; one failure doesn't kill the run	One exception crashes everything; shared mutable state
4	Tool allowlist	Explicit list; tools minimal and documented	Generic 'shell' tool; tools added ad hoc
5	Path guard	All paths resolved inside workspace/; ../ rejected	Absolute paths accepted; writes to home dir
6	Human approval	Risky actions listed; approval prompts logged with decision	No list of risky actions; AUTO_APPROVE left on
7	Sandbox config	Isolated, time-limited, no network unless needed, resource limits	Code runs on host; internet + secrets available inside
8	Checker	Machine-checkable expectations per task	“Looked right to me”
9	Suite pass rate	N tasks × ≥3 runs, mean and spread, cost per task	One successful demo run
10	Step log	Every model call, tool call and result with tokens and cost	Only final answer kept; no way to replay

Project 02 Long-horizon agents

⏱️ 6–10 hours | 💶 < $2 LLM credit + E2B free credit | 🧰 IDE track strongly recommended

Goal. An agent that takes a multi-step task ("write a script, run it, save the result, write a report"), plans it, hands sub-tasks to **parallel sub-agents**, runs code **only inside a sandbox**, stays inside **hard limits** (steps, tool calls, money), asks a **human before risky actions**, and leaves a **JSON log** of every step. A small task set measures how often it succeeds.

task → planner → 1–4 subtasks (typed) → workers in parallel (asyncio) → tools: search / read / write / run_python→E2B → writer merg

Guardrails in this build

Risk	Control	Where
Runaway loops	`UsageLimits(request_limit=MAX_STEPS, tool_calls_limit=…)` → raises `UsageLimitExceeded`	agent.py `logged_run`
Cost blow-up	`cost_limit=Decimal(MAX_COST_USD)` (best-effort: needs pricing data for the model) + OpenRouter key credit limit (hard)	agent.py + OpenRouter key settings
Agent uses tools it shouldn't	Explicit allowlist `ALLOWED_TOOLS`; tools outside it are never registered	agent.py `_tools()`
Agent damages files	Path guard: only `workspace/`; overwriting an existing file raises `ApprovalRequired` → human y/N	`_safe`, `write_file`, `ask_human`
Malicious/buggy code	Runs in E2B micro-VM (5-min lifetime, internet off) or Docker (no network, 512 MB, 1 CPU, read-only)	sandbox.py
No audit trail	Every model call, tool call and tool result → JSONL with tokens and cost	logger.py

Step-by-step

1 Folder, requirements, keys

requirements.txt

```
# Project 02 - long-horizon agents (written against pydantic-ai 2.63, e2b-code-interpreter 2.x)
pydantic-ai-slim[openrouter]>=2.63,<3
e2b-code-interpreter>=2.0
python-dotenv>=1.0
```

.env.example

```
OPENROUTER_API_KEY=sk-or-v1-paste-your-key-here
E2B_API_KEY=e2b_paste-your-key-here
AGENT_MODEL=openai/gpt-4o-mini
SANDBOX=e2b        # or: docker
MAX_STEPS=25       # model requests per agent run
MAX_COST_USD=0.50  # per run, best-effort (see guide)
```

Create the venv and install as in Part 0 (B4). `pydantic-ai-slim[openrouter]` installs Pydantic AI with just the OpenRouter provider. Create an empty `workspace/` folder.

> **Version pins matter.** Pydantic AI moves fast. Everything here was checked against **pydantic-ai 2.63.0** and **e2b-code-interpreter 2.11.6** (latest on 30 Oct 2026). If you install a newer major version and imports fail, pin: `pip install "pydantic-ai-slim[openrouter]==2.63.0"`.

2 sandbox.py — where code is allowed to run

Two back-ends behind one function. E2B: a fresh cloud micro-VM per call, killed in `finally`. Docker: a throwaway container with no network.

sandbox.py

```python
"""Run untrusted, model-written Python somewhere that is NOT your laptop."""
import os
import pathlib
import subprocess
import tempfile


def run_in_e2b(code: str, timeout: int = 60) -> str:
    from e2b_code_interpreter import Sandbox  # needs E2B_API_KEY in the environment
    # isolated microVM; lifetime capped at 5 min, no internet from inside
    sbx = Sandbox.create(timeout=300, allow_internet_access=False)
    try:
        ex = sbx.run_code(code, timeout=timeout)
        out = "".join(ex.logs.stdout) + "".join(ex.logs.stderr)
        if ex.error:
            out += f"\nERROR {ex.error.name}: {ex.error.value}"
        return out[-4000:] or "(no output)"
    finally:
        sbx.kill()                              # always destroy the VM


def run_in_docker(code: str, timeout: int = 60) -> str:
    """Throwaway container: no network, 512 MB RAM, 1 CPU, read-only code mount."""
    with tempfile.TemporaryDirectory() as d:
        pathlib.Path(d, "main.py").write_text(code)
        cmd = ["docker", "run", "--rm", "--network", "none", "--memory", "512m",
               "--cpus", "1", "--pids-limit", "128", "-v", f"{d}:/work:ro",
               "python:3.11-slim", "python", "/work/main.py"]
        try:
            p = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
        except subprocess.TimeoutExpired:
            return "ERROR: timed out"
        return ((p.stdout + p.stderr)[-4000:]) or "(no output)"


def run_code(code: str) -> str:
    backend = os.getenv("SANDBOX", "e2b").lower()
    return run_in_docker(code) if backend == "docker" else run_in_e2b(code)
```

Terminal

```
# smoke-test the sandbox alone
python -c "from sandbox import run_code; print(run_code('print(2+2)'))"          # E2B -> 4
SANDBOX=docker python -c "from sandbox import run_code; print(run_code('print(2+2)'))"
```

3 logger.py — one JSON line per step

logger.py

```python
"""JSON-lines step log: one line per tool call and per model response."""
import json
import pathlib
import time
from datetime import datetime, timezone

LOG_DIR = pathlib.Path("logs")


class StepLogger:
    def __init__(self, run_name: str):
        LOG_DIR.mkdir(exist_ok=True)
        stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
        self.path = LOG_DIR / f"{stamp}_{run_name}.jsonl"
        self.step = 0

    def log(self, **event):
        self.step += 1
        event = {"step": self.step, "ts": time.time(), **event}
        with self.path.open("a") as f:
            f.write(json.dumps(event, default=str) + "\n")
        print(f"  step {self.step:>3} {event.get('kind'):>9} {event.get('tool', '')}")
```

4 agent.py — tools, guardrails, planner + parallel workers

Read it top to bottom; each guardrail from the table above is commented in the code.

agent.py

```python
"""Long-horizon agent: plan -> parallel sub-agents -> writer, with guardrails.

  python agent.py "Research what HNSW is and write report.md with a small Python benchmark"
"""
import asyncio
import os
import pathlib
import re
import sys
from decimal import Decimal

from dotenv import load_dotenv
from pydantic import BaseModel, Field
from pydantic_ai import (Agent, ApprovalRequired, ModelRetry, DeferredToolRequests,
DeferredToolResults, RunContext,
                         Tool, ToolDenied, UsageLimitExceeded, UsageLimits)
from pydantic_ai.capabilities import HandleDeferredToolCalls
from pydantic_ai.messages import ModelResponse, ToolCallPart, ToolReturnPart

from logger import StepLogger
from sandbox import run_code

load_dotenv()
os.environ.setdefault("PYDANTIC_AI_NO_BANNER", "1")
WORKSPACE = pathlib.Path("workspace").resolve()
WORKSPACE.mkdir(exist_ok=True)
MAX_STEPS = int(os.getenv("MAX_STEPS", "25"))
MAX_COST = Decimal(os.getenv("MAX_COST_USD", "0.50"))
ALLOWED_TOOLS = {"search_files", "read_file", "write_file", "run_python"}  # allowlist


def get_model():
    name = os.getenv("AGENT_MODEL", "openai/gpt-4o-mini")
    if name == "test":                        # offline dry run, no API key, no cost
        from pydantic_ai.models.test import TestModel
        return TestModel()
    from pydantic_ai.models.openrouter import OpenRouterModel
    return OpenRouterModel(name)              # reads OPENROUTER_API_KEY


# ---------- typed tools (Pydantic validates every argument) ----------
def _safe(path: str) -> pathlib.Path:
    p = (WORKSPACE / path).resolve()
    if WORKSPACE not in p.parents and p != WORKSPACE:
        raise ValueError("path escapes the workspace")   # guardrail: no ../../etc
    return p


def search_files(query: str) -> list[str]:
    """Search the workspace for files whose text contains the query (case-insensitive)."""
    hits = []
    for f in WORKSPACE.rglob("*"):
        if f.is_file() and query.lower() in f.read_text(errors="ignore").lower():
            hits.append(str(f.relative_to(WORKSPACE)))
    return hits[:20] or ["no matches"]


def read_file(path: str) -> str:
    """Read a text file from the workspace (max 8000 characters)."""
    p = _safe(path)
    if not p.is_file():
        raise ModelRetry(f"{path} does not exist; call search_files first")  # model retries
    return p.read_text(errors="ignore")[:8000]


def write_file(ctx: RunContext, path: str, content: str) -> str:
    """Create or overwrite a text file in the workspace."""
```

```
    p = _safe(path)
    if p.exists() and not ctx.tool_call_approved:
        raise ApprovalRequired          # overwriting = risky -> ask a human first
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content)
    return f"wrote {len(content)} chars to {path}"


def run_python(code: str) -> str:
    """Execute Python code in an isolated sandbox (E2B or Docker) and return stdout/stderr."""
    if re.search(r"\b(rm -rf|shutil\.rmtree|os\.remove)\b", code):
        return "blocked: destructive operation"          # cheap static guardrail
    try:
        return run_code(code)
    except Exception as e:                               # report, don't crash the run
        return f"sandbox error: {type(e).__name__}: {e}"


def _tools():
    all_tools = [Tool(search_files), Tool(read_file), Tool(write_file), Tool(run_python)]
    return [t for t in all_tools if t.name in ALLOWED_TOOLS]


async def ask_human(ctx: RunContext, requests: DeferredToolRequests) -> DeferredToolResults:
    approvals = {}
    for call in requests.approvals:
        if os.getenv("AUTO_APPROVE") == "1":
            approvals[call.tool_call_id] = True
            continue
        ans = await asyncio.to_thread(input, f"\nApprove {call.tool_name}({call.args})? [y/N] ")
        approvals[call.tool_call_id] = True if ans.strip().lower() == "y" else ToolDenied("Human said no")
    return requests.build_results(approvals=approvals)


# ---------- agents ----------
class Plan(BaseModel):
    subtasks: list[str] = Field(min_length=1, max_length=4,
                                description="independent subtasks that can run in parallel")


planner = Agent(get_model(), output_type=Plan,
                instructions="Split the task into 1-4 independent subtasks. Do not do them.")

worker = Agent(get_model(), tools=_tools(),
               capabilities=[HandleDeferredToolCalls(handler=ask_human)],
               instructions=("You complete ONE subtask using tools. Save findings to "
                             "workspace files named notes_<topic>.md. Use run_python to verify "
                             "any code. Finish with a 3-line summary."))

writer = Agent(get_model(), tools=_tools(),
               capabilities=[HandleDeferredToolCalls(handler=ask_human)],
               instructions="Read the notes_*.md files and write the final deliverable requested.")


async def logged_run(agent: Agent, prompt: str, log: StepLogger, name: str):
    """agent.iter() lets us see every model response and tool result as it happens."""
    limits = UsageLimits(request_limit=MAX_STEPS, tool_calls_limit=MAX_STEPS * 2,
                         cost_limit=MAX_COST)
    async with agent.iter(prompt, usage_limits=limits) as run:
        async for node in run:
            if Agent.is_call_tools_node(node):
                resp: ModelResponse = node.model_response
                u = resp.usage
                log.log(kind="model", agent=name, model=resp.model_name,
                        input_tokens=u.input_tokens, output_tokens=u.output_tokens,
                        cost_usd=getattr(u, "cost", None))
                for part in resp.parts:
                    if isinstance(part, ToolCallPart):
                        log.log(kind="call", agent=name, tool=part.tool_name, input=part.args)
            elif Agent.is_model_request_node(node):
                for part in node.request.parts:
                    if isinstance(part, ToolReturnPart):
                        log.log(kind="result", agent=name, tool=part.tool_name,
```

```
                                output=str(part.content)[:500])
        return run.result.output, run.usage


async def main(task: str):
    log = StepLogger("run")
    plan, _ = await logged_run(planner, task, log, "planner")
    print("plan:", plan.subtasks)
    # sub-agents in parallel; one failure doesn't kill the others
    results = await asyncio.gather(
        *[logged_run(worker, f"Overall task: {task}\nYour subtask: {s}", log, f"worker{i}")
          for i, s in enumerate(plan.subtasks)],
        return_exceptions=True)
    for s, r in zip(plan.subtasks, results):
        status = "ERROR " + repr(r) if isinstance(r, Exception) else "ok"
        print(f"- {s[:60]}: {status}")
    final, usage = await logged_run(writer, f"Task: {task}", log, "writer")
    print("\nFINAL:\n", final)
    print(f"\nlog file: {log.path}")


if __name__ == "__main__":
    try:
        asyncio.run(main(" ".join(sys.argv[1:]) or "Write hello.md that says hello"))
    except UsageLimitExceeded as e:
        print("STOPPED by budget/step guardrail:", e)
```

How the pieces map to Pydantic AI 2.x: `Tool(fn)` turns a typed Python function into a tool (the type hints + docstring become the JSON schema the model sees). `output_type=Plan` forces a validated Pydantic object. `raise ModelRetry(...)` sends an error back to the model to try again. `raise ApprovalRequired` pauses the tool call; the `HandleDeferredToolCalls` capability calls `ask_human`, which returns approvals or `ToolDenied`. `agent.iter()` lets us see every node to log it.

5 Run it offline first (no keys, no cost)

`AGENT_MODEL=test` swaps in Pydantic AI's `TestModel`, which calls every tool with dummy arguments — it checks the wiring, logging and approvals, not intelligence.

macOS (Terminal)

```
AGENT_MODEL=test AUTO_APPROVE=1 python agent.py "test task"
```

Windows PowerShell

```
$env:AGENT_MODEL="test"; $env:AUTO_APPROVE="1"
python agent.py "test task"
```

You should see numbered steps (`model`, `call`, `result`), a FINAL block and `log file: logs/…_run.jsonl`. In my offline test run (no key set) the E2B call correctly reported "API key is required" because no key was set — the agent caught it and carried on.

6 Run it for real

Terminal

```
python agent.py "Write fizzbuzz.py that prints FizzBuzz for 1..15, run it in the sandbox, and save the output to fizzbuzz_output.txt"
```

When a tool wants to overwrite a file you'll get `Approve write_file({...})? [y/N]`. Then open the log:

logs/…_run.jsonl (shape; your numbers will differ)

```
{"step": 3, "kind": "model", "agent": "worker0", "model": "openai/gpt-4o-mini", "input_tokens": 512, "output_tokens": 61, "cost_usd": 0.0001}
{"step": 4, "kind": "call", "agent": "worker0", "tool": "run_python", "input": {"code": "for i in range(1,16): ..."}}
```

7 tests/tasks.json + run_tests.py — a pass rate for agents

Each task has machine-checkable expectations (file exists, content matches a regex). The runner resets `workspace/` before each task.

tests/tasks.json

```
[
  {"task": "Write fizzbuzz.py that prints FizzBuzz for 1..15, run it in the sandbox, and save the
output to fizzbuzz_output.txt",
   "checks": [{"file": "fizzbuzz.py"}, {"file": "fizzbuzz_output.txt", "regex": "FizzBuzz"}]},
  {"task": "Compute the first 10 prime numbers with Python in the sandbox and write them comma-
separated to primes.txt",
   "checks": [{"file": "primes.txt", "regex": "2,\\s*3,\\s*5,\\s*7,\\s*11"}]},
  {"task": "Write report.md explaining chunk size vs overlap in RAG with a markdown table of 3
settings",
   "checks": [{"file": "report.md", "regex": "\\|.*overlap.*\\|"}]}
]
```

run_tests.py

```
"""Run the agent on every task in tests/tasks.json and check the files it produced.

  AUTO_APPROVE=1 python run_tests.py
"""
import asyncio
import json
import pathlib
import re
import shutil

import agent

tasks = json.loads(pathlib.Path("tests/tasks.json").read_text())
passed = 0
for i, t in enumerate(tasks, 1):
    shutil.rmtree(agent.WORKSPACE, ignore_errors=True)
    agent.WORKSPACE.mkdir()
    print(f"\n=== task {i}: {t['task'][:70]}")
    try:
        asyncio.run(agent.main(t["task"]))
    except Exception as e:  # budget exceeded, API error, ...
        print("run failed:", e)
    ok = True
    for c in t["checks"]:
        f = agent.WORKSPACE / c["file"]
        if not f.exists() or ("regex" in c and not re.search(c["regex"], f.read_text(), re.I)):
            ok = False
            print("  check failed:", c)
    passed += ok
    print("  PASS" if ok else "  FAIL")
print(f"\ntask pass rate: {passed}/{len(tasks)} = {passed / len(tasks):.0%}")
```

Terminal

```
AUTO_APPROVE=1 python run_tests.py     # prints: task pass rate: x/3 = …%
```

Make it yours: add 5–10 tasks from your world (e.g. “read sales.csv, compute churn by segment, write report.md”). Run each task 3× — agents are non-deterministic, so report the success rate and average cost per task, not one lucky run.

Done when

- ☐ The agent completes a multi-step task end-to-end with code executed only in the sandbox
- ☐ Sub-agents run in parallel and the writer merges their results
- ☐ Hard caps on steps / tool calls / cost stop a runaway run (try MAX_STEPS=2 and watch it stop)
- ☐ A risky action (overwrite) waits for human approval; a denied action is handled gracefully

☐ Every step is in a JSONL log, and `run_tests.py` prints a pass rate you've put in the README

Common errors & fixes

Symptom	Fix
`UserError: ... OPENROUTER_API_KEY`	Key missing in `.env` / not loaded; run from the `02-agents` folder.
`UsageLimitExceeded`	Working as designed. Raise MAX_STEPS or simplify the task; check the log for loops.
`CostNotFoundWarning`	Pydantic AI has no price for that model id, so the cost cap can't be enforced → rely on the OpenRouter key credit limit, or `pydantic_ai.prices.update_in_background()`.
`AuthenticationException` from E2B	`E2B_API_KEY` missing → add it, or set `SANDBOX=docker`.
Docker: `Cannot connect to the Docker daemon`	Start Docker Desktop and wait for "Engine running".
Model never calls tools / invents results	Pick a model with good tool calling (gpt-4o-mini, Claude Sonnet, Gemini Flash); keep tool docstrings short and specific.
`ImportError` on a pydantic_ai name	Different major version installed → `pip show pydantic-ai-slim`; pin 2.63.0.
Two approval prompts at once, input gets mixed	Workers run in parallel; use AUTO_APPROVE for tests, or set the worker concurrency to 1 while demoing approvals.

02 · Evals Agents — evals, worked example

What a reviewed agent run looks like. The log format is the real one from `logger.py`; the values are `illustrative values` (a dry run with the offline test model produced the same event sequence).

Step log rendered as a timeline

1. MODEL planner `openai/gpt-4o-mini` in 412 · out 58 · $0.0001
2. CALL planner `final_result` {subtasks:[write fizzbuzz.py, run + save output]}
3. MODEL worker0 `openai/gpt-4o-mini` in 690 · out 141
4. CALL worker0 `write_file` {path:'fizzbuzz.py', content:'for i in range(1,16)...'}
5. CALL worker1 `run_python` {code:'for i in range(1,16): ...'}
6. RESULT worker1 `run_python` stdout: 1 2 Fizz 4 Buzz ... FizzBuzz
7. CALL worker1 `write_file` {path:'fizzbuzz_output.txt'} → exists → APPROVAL y
8. RESULT writer `final` Created fizzbuzz.py and fizzbuzz_output.txt

logs/…_run.jsonl — one raw line

```
{"step": 4, "ts": 1793341790.1, "kind": "call", "agent": "worker0", "tool": "write_file", "input": {"path": "fizzbuzz.py", "content": "..."}}
```

Test-suite table, 3 runs per task `illustrative values`

Task (tests/ tasks.json)	Checks	Run 1	Run 2	Run 3	Pass	Avg steps	Avg $
fizzbuzz.py + output file	file exists · output ∋ “FizzBuzz”	✔	✔	✔	3/3	9	0.002
primes below 50 → primes.txt	file ∋ “47”	✔	✘ wrong file name	✔	2/3	11	0.003
report.md with a table	file ∋ “|”	✘ UsageLimitExceeded	✔	✔	2/3	18	0.006
Suite — report: 7/9 runs = 78%, task-level 1/3 always passing					**7/9**	13	0.004

Read it like this: the headline is 7/9, but the useful facts are *which* tasks are flaky and *why* (a naming mismatch, a limit hit). Always report variance across runs.

02 · Review

Agents — reviewer's checklist

Ask the engineer	Artefact to demand	Good answer	Red flag
Show me the step log of a failed run.	logs/*.jsonl for a FAIL	Can replay what the agent saw and did, step by step	No logs, or only final outputs
Which tools can it call, and why each?	ALLOWED_TOOLS + docstrings	Minimal set; each tool has a narrow purpose	Shell/HTTP 'do anything' tools
How is code isolated?	Sandbox config (E2B template / docker flags)	No network by default, CPU/ mem/time limits, no secrets inside	Runs on the developer's machine or a shared server
What are the caps?	MAX_STEPS, tool-call cap, cost cap, provider credit limit	Values + a test proving they stop a loop	“We'll watch it”
Failure and retry stats?	Counts of ModelRetry, tool errors, limit hits per suite	Known top-3 failure modes with fixes	Unknown; failures re-run until green
Where does a human approve?	List of risky actions + approval log	Irreversible/external actions always gated	Approval only in the demo, AUTO_APPROVE in prod
Pass rate and variance?	Suite results over ≥3 runs	Mean ± spread, per-task breakdown, cost per task	A single run or a hand-picked demo
Which model and versions?	requirements pins, model id	Pinned library versions; model id in every log line	Floating 'latest' everywhere

02 · Errors

Agents — error gallery

Snippets are `illustrative values` (representative log lines and exception types).

Agent loops on the same tool call

```
step 14 call search_files {"query":"report"}
step 16 call search_files {"query":"report"}
step 18 call search_files {"query":"report"}
…
UsageLimitExceeded: request_limit of 25
```

Symptom	Run ends with a limit error after many identical steps.
How to spot	Repeated identical 'call' lines in the JSONL log.
Likely causes	Tool returns nothing useful; model doesn't know how to proceed; vague task.
Fix	Return actionable messages (ModelRetry with a hint); dedupe identical calls; clearer task.

Writing outside the workspace

```
call write_file {"path": "../../.ssh/config", …}
→ ValueError: path escapes workspace
```

Symptom	Agent tries to touch files it shouldn't.
How to spot	Path-guard errors in the log; any path with ../ or absolute paths.
Likely causes	Prompt injection in a file it read, or a confused model.
Fix	Keep _safe() on every file tool; treat a hit as a security event; review inputs.

Structured output validation failure

```
planner → Plan{subtasks: [7 items]}
ValidationError: List should have at most 4 items
UnexpectedModelBehavior: Exceeded maximum
  retries for output validation
```

Symptom	Run stops before any work starts.
How to spot	Planner step shows retries then an exception.
Likely causes	Schema stricter than the instructions; weak model; long task.
Fix	State the limits in instructions; allow retries; simplify the task or the schema.

Cost blow-up

```
worker0 model in 48,210 tokens  $0.0072
worker0 model in 51,880 tokens  $0.0078
… 40 requests …  total $0.31 for one task
```

Symptom	One task costs 50× the others.
How to spot	cost_usd and input_tokens climbing per step in the log.
Likely causes	Whole files pasted back into history each turn; no cost cap; big model for small steps.
Fix	Truncate tool outputs, cost cap + provider credit limit, smaller model for workers.

Sandbox can't do what the code needs

```
result run_python:
ModuleNotFoundError: No module named 'pandas'
# or: timeout after 300 s / network unreachable
```

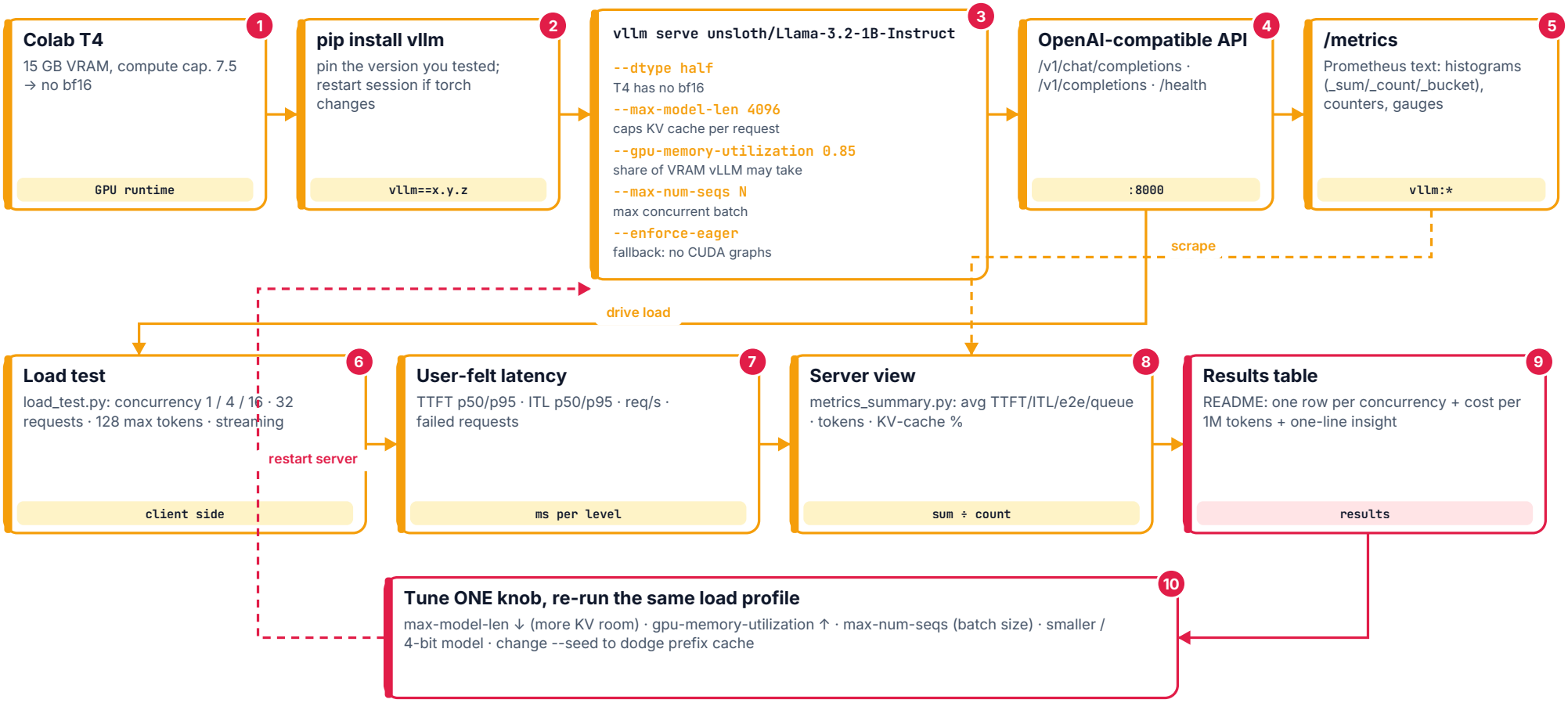
Symptom	Code works locally, fails in the sandbox.
How to spot	stderr in 'result' events.
Likely causes	Missing package, no internet by design, timeout too short.
Fix	Custom sandbox template with deps pre-installed; explicit, small timeouts.

Flaky pass rate reported once

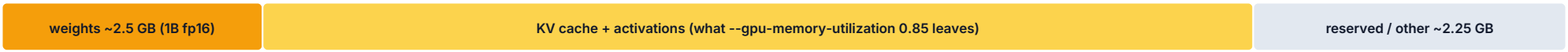
```
run A: 3/3 PASS   ← in the deck
run B: 1/3 PASS
run C: 2/3 PASS
```

Symptom	Demo works, pilot doesn't.
How to spot	Ask for ≥3 runs per task and the spread.
Likely causes	Non-deterministic models; temperature; tasks on the edge of capability.
Fix	Report mean ± spread, fix the flakiest task first, temperature 0 where possible.

03 · Flow Deploy — detailed flow with checkpoints



T4 MEMORY BUDGET — illustrative, from the 2 GB/B-param rule of thumb (not measured)



0 GB 15 GB

an 8B model in fp16 (~16 GB) does not fit at all → 4-bit AWQ or a bigger GPU

03 · Checkpoints

Deploy — checkpoints

Checkpoint guide — Project 03

#	Checkpoint — what to inspect	Good looks like	Red flags
1	GPU and runtime	GPU type, VRAM and compute capability written in the README	“It ran on Colab” — which GPU?
2	Versions pinned	vllm==x.y.z, model repo + revision	Unpinned; results not reproducible next week
3	Serve flags	Every flag explained (dtype, max-model-len, memory util, max-num-seqs)	Copied flags nobody can explain
4	Endpoint health	/health ok, a curl example in the README	Only tested from inside a notebook
5	Metrics source	/metrics scraped; metric names listed	Only client stopwatch numbers
6	Load profile	Concurrency levels, request count, prompt/output lengths, seed	“We sent some requests”
7	User-felt latency	TTFT p50 and p95, ITL p50/p95 per level	Averages only; no tail latency
8	Server view agrees	Server TTFT ≈ client TTFT minus network	Big unexplained gap
9	Results table	One row per level + error rate + cost per 1M tokens	Single best number
10	Tuning	One knob at a time, before/after	Several changes, best run only

Project 03 Deploy & measure a model

⏱️ 2–4 hours | 💶 €0 on Colab free T4 | 🧰 Colab (GPU) + any laptop for the client scripts

Goal. Serve an open model behind an OpenAI-compatible API with **vLLM**, then measure what users feel — **TTFT** (time to first token), **inter-token latency** (smoothness of streaming), and **throughput** (tokens/s for everyone) — as concurrent users grow. Publish a results table.

Will it fit? (carousel table)

Model	Params	fp16 memory	Free T4 (15 GB)?
Llama-3.2-1B	1B	~2.5 GB	✅ yes — start here
Qwen3-4B	4B	~8 GB	✅ yes
any 8B model	8B	~16 GB	⚠️ only 4-bit (AWQ)

Rule of thumb: ~2 GB per billion parameters in fp16, *plus* the KV cache (grows with context length × concurrent requests). The T4 has no bf16 support, hence `--dtype half`.

Step-by-step (Colab)

1. **Switch to a GPU runtime**

   Runtime → Change runtime type → T4 GPU → Save. Free sessions are limited (up to ~12 h, and can disconnect when idle) — keep this project short.

2 Paste these cells one by one

colab_cells.py

```
# ==== Google Colab cells for Project 03. Runtime > Change runtime type > T4 GPU > Save ====

# --- Cell 1: check the GPU
!nvidia-smi

# --- Cell 2: install vLLM (takes a few minutes; if Colab asks to restart the session, do it,
#             then continue from Cell 3)
!pip install -q vllm httpx

# --- Cell 3: start the server in the background, logs go to vllm.log
!nohup vllm serve unsloth/Llama-3.2-1B-Instruct --dtype half --max-model-len 4096 \
    --gpu-memory-utilization 0.85 --port 8000 > vllm.log 2>&1 &

# --- Cell 4: wait until it is healthy (first start downloads ~2.5 GB of weights)
import time, httpx
for _ in range(120):
    try:
        if httpx.get("http://localhost:8000/health", timeout=2).status_code == 200:
            print("server up"); break
    except Exception:
        pass
    time.sleep(5)
else:
    print(open("vllm.log").read()[-3000:])   # show the error if it never came up

# --- Cell 5: first request
!curl -s localhost:8000/v1/chat/completions -H "Content-Type: application/json" \
  -d '{"model":"unsloth/Llama-3.2-1B-Instruct","messages":[{"role":"user","content":"Say hi in 5
words"}],"max_tokens":20}'

# --- Cell 6: raw metrics (TTFT, inter-token latency, tokens...)
!curl -s localhost:8000/metrics | grep -E "^vllm:(time_to_first_token|inter_token_latency|
generation_tokens|num_requests)" | head -30

# --- Cell 7: upload load_test.py + metrics_summary.py (folder icon on the left), then:
!python load_test.py --concurrency 1 4 16 --requests 32 --max-tokens 128
!python metrics_summary.py

# --- Cell 8 (optional): vLLM's own benchmark
!vllm bench serve --backend vllm --model unsloth/Llama-3.2-1B-Instruct --endpoint /v1/completions \
    --dataset-name random --num-prompts 100 --random-input-len 512 --random-output-len 128 --seed 1
```

Unverified on today's versions: Not yet run on a GPU. Current vLLM is 0.32.0. Recent vLLM releases target newer GPUs first; if the server fails to start on the T4 (look at `vllm.log`), try in order: add `--enforce-eager`; lower `--max-model-len 2048` and `--gpu-memory-utilization 0.8`; pin an older vLLM that your Colab image supports; or switch to an L4/A10G on Lightning AI (Step 7).

3 Understand the server metrics

vLLM exposes Prometheus metrics at `/metrics`. The ones that matter:

Metric	Type	Meaning
`vllm:time_to_first_token_seconds`	histogram	TTFT: queue + prefill time
`vllm:inter_token_latency_seconds`	histogram	gap between streamed tokens
`vllm:e2e_request_latency_seconds`	histogram	whole request
`vllm:request_queue_time_seconds`	histogram	waiting before scheduling
`vllm:generation_tokens_total` / `vllm:prompt_tokens_total`	counter	tokens out / in (throughput = Δtokens / Δtime)
`vllm:num_requests_running` / `_waiting`, `vllm:kv_cache_usage_perc`	gauge	load and memory pressure right now

Histograms come as `_sum`, `_count`, `_bucket`: average = sum ÷ count.

metrics_summary.py

```
"""Read vLLM's Prometheus /metrics and print server-side averages.

  python metrics_summary.py --url http://localhost:8000
Histogram metrics expose <name>_sum and <name>_count; average = sum / count.
"""
import argparse
import re

import httpx

HISTOGRAMS = {
    "vllm:time_to_first_token_seconds": "TTFT",
    "vllm:inter_token_latency_seconds": "inter-token latency",
    "vllm:e2e_request_latency_seconds": "end-to-end latency",
    "vllm:request_queue_time_seconds": "queue time",
}
COUNTERS = {"vllm:prompt_tokens_total": "prompt tokens",
            "vllm:generation_tokens_total": "generated tokens",
            "vllm:request_success_total": "finished requests"}
GAUGES = {"vllm:num_requests_running": "running now", "vllm:num_requests_waiting": "waiting now",
          "vllm:kv_cache_usage_perc": "KV cache used (0-1)"}


def total(text: str, name: str) -> float:
    """Sum a metric across all label sets, e.g. vllm:x_sum{model_name="..."} 1.23"""
    pat = re.compile(rf"^{re.escape(name)}(?:{{[^}}]*}})?\s+([0-9.eE+-]+)$", re.M)
    return sum(float(v) for v in pat.findall(text))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", default="http://localhost:8000")
    text = httpx.get(f"{ap.parse_args().url}/metrics", timeout=10).text
    for name, label in HISTOGRAMS.items():
        s, c = total(text, name + "_sum"), total(text, name + "_count")
        print(f"{label:>22}: " + (f"avg {1000 * s / c:.1f} ms over {int(c)} samples" if c else
"no data yet"))
    for name, label in {**COUNTERS, **GAUGES}.items():
        print(f"{label:>22}: {total(text, name):g}")


if __name__ == "__main__":
    main()
```

4 load_test.py — what users feel at 1, 4, 16 concurrent users

Client-side measurement over streaming `/v1/chat/completions`. Works against vLLM, SGLang or any OpenAI-compatible server.

load_test.py

```
"""Async load test for any OpenAI-compatible server (vLLM, SGLang, ...).
Measures, per request, from the client side:
  TTFT  = time to first streamed token
  ITL   = gaps between streamed chunks (inter-token latency)
  and overall output tokens/second (throughput).

  python load_test.py --url http://localhost:8000 --model unsloth/Llama-3.2-1B-Instruct \
      --concurrency 1 4 16 --requests 32 --max-tokens 128
"""
import argparse
import asyncio
import json
import statistics
import time

import httpx

PROMPT = "Explain retrieval-augmented generation to a CFO in about 150 words."


def pct(xs, p):
    xs = sorted(xs)
    return xs[min(len(xs) - 1, int(round(p / 100 * (len(xs) - 1))))] if xs else float("nan")


async def one_request(client, url, model, max_tokens):
    body = {"model": model, "stream": True, "max_tokens": max_tokens, "temperature": 0,
            "messages": [{"role": "user", "content": PROMPT}],
            "stream_options": {"include_usage": True}}
    t0 = time.perf_counter()
    ttft, last, gaps, chunks, usage_tokens = None, None, [], 0, None
    async with client.stream("POST", f"{url}/v1/chat/completions", json=body) as r:
        r.raise_for_status()
        async for line in r.aiter_lines():
            if not line.startswith("data: "):
                continue
            data = line[6:]
            if data.strip() == "[DONE]":
                break
            msg = json.loads(data)
            if msg.get("usage"):
                usage_tokens = msg["usage"].get("completion_tokens")
            if not msg.get("choices") or not msg["choices"][0]["delta"].get("content"):
                continue
            now = time.perf_counter()
            if ttft is None:
                ttft = now - t0
            else:
                gaps.append(now - last)
            last = now
            chunks += 1
    return {"ttft": ttft, "gaps": gaps, "e2e": time.perf_counter() - t0,
            "tokens": usage_tokens or chunks}


async def run_level(url, model, concurrency, n, max_tokens):
    sem = asyncio.Semaphore(concurrency)
    async with httpx.AsyncClient(timeout=300) as client:
        async def guarded():
            async with sem:
                return await one_request(client, url, model, max_tokens)
        t0 = time.perf_counter()
        res = await asyncio.gather(*[guarded() for _ in range(n)], return_exceptions=True)
        wall = time.perf_counter() - t0
    ok = [r for r in res if isinstance(r, dict) and r["ttft"] is not None]
```

```
    ttfts = [r["ttft"] * 1000 for r in ok]
    itls = [g * 1000 for r in ok for g in r["gaps"]]
    toks = sum(r["tokens"] for r in ok)
    return {"concurrency": concurrency, "ok": len(ok), "failed": n - len(ok),
            "ttft_p50_ms": statistics.median(ttfts) if ttfts else float("nan"),
            "ttft_p95_ms": pct(ttfts, 95),
            "itl_p50_ms": statistics.median(itls) if itls else float("nan"),
            "itl_p95_ms": pct(itls, 95),
            "throughput_tok_s": toks / wall, "req_s": len(ok) / wall}


async def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", default="http://localhost:8000")
    ap.add_argument("--model", default="unsloth/Llama-3.2-1B-Instruct")
    ap.add_argument("--concurrency", type=int, nargs="+", default=[1, 4, 16])
    ap.add_argument("--requests", type=int, default=32)
    ap.add_argument("--max-tokens", type=int, default=128)
    a = ap.parse_args()
    rows = [await run_level(a.url, a.model, c, a.requests, a.max_tokens) for c in a.concurrency]
    print("| concurrency | ok/failed | TTFT p50 (ms) | TTFT p95 (ms) | ITL p50 (ms) | ITL p95 (ms) | output
tok/s | req/s |")
    print("|---|---|---|---|---|---|---|---|")
    for r in rows:
        print(f"| {r['concurrency']} | {r['ok']}/{r['failed']} | {r['ttft_p50_ms']:.0f} | {r['ttft_p95_ms']:.
0f} "
              f"| {r['itl_p50_ms']:.1f} | {r['itl_p95_ms']:.1f} | {r['throughput_tok_s']:.0f} | {r['req_s']:.
2f} |")


if __name__ == "__main__":
    asyncio.run(main())
```

Real output against the included fake server (so you know the format — **these are not GPU numbers**):

output (fake server)

```
| concurrency | ok/failed | TTFT p50 (ms) | TTFT p95 (ms) | ITL p50 (ms) | ITL p95 (ms) | output tok/s | req/
s |
|---|---|---|---|---|---|---|---|
| 1 | 16/0 | 53 | 54 | 10.2 | 10.5 | 84 | 2.63 |
| 8 | 16/0 | 58 | 59 | 10.3 | 10.7 | 655 | 20.45 |
```

5 Test the scripts without a GPU (optional)

`fake_server.py` mimics vLLM's streaming API and `/metrics` so you can debug the client on your laptop (`pip install aiohttp httpx`).

fake_server.py

```
"""Tiny fake OpenAI-compatible streaming server + /metrics, to test the scripts without a GPU.
  python fake_server.py   (then run load_test.py --url http://localhost:8001)"""
import asyncio, json
from aiohttp import web

async def chat(req):
    body = await req.json()
    resp = web.StreamResponse(headers={"Content-Type": "text/event-stream"})
    await resp.prepare(req)
    await asyncio.sleep(0.05)
    n = body.get("max_tokens", 16)
    for i in range(n):
        chunk = {"choices": [{"delta": {"content": f"tok{i} "}}]}
        await resp.write(f"data: {json.dumps(chunk)}\n\n".encode()); await asyncio.sleep(0.01)
    await resp.write(f"data: {json.dumps({'choices': [], 'usage': {'completion_tokens': n}})}\n\n".enc
ode())
    await resp.write(b"data: [DONE]\n\n")
    return resp

async def metrics(req):
    return web.Response(text='vllm:time_to_first_token_seconds_sum{model_name="m"} 1.5\n'
                             'vllm:time_to_first_token_seconds_count{model_name="m"} 30\n'
                             'vllm:generation_tokens_total{model_name="m"} 3840\n')

app = web.Application(); app.router.add_post("/v1/chat/completions", chat); app.router.add_get("/
metrics", metrics)
web.run_app(app, port=8001)
```

Terminal

```
python fake_server.py &            # Windows: run it in a second terminal
python load_test.py --url http://localhost:8001 --model x --concurrency 1 8 --requests 16 --max-tokens
32
```

6 vLLM's built-in benchmark (cross-check)

`vllm bench serve` reports request throughput, output tokens/s and mean/median/P99 of TTFT, TPOT (time per output token) and ITL. Use `--dataset-name random` (or `sharegpt` after downloading it). vLLM caches prefixes, so change `--seed` between runs to avoid flattering numbers. vLLM's docs also point to GuideLLM for heavier load testing.

7 Alternatives: SGLang, Lightning AI

SGLang (0.5.23) is the other popular fast server, also OpenAI-compatible, so `load_test.py` works unchanged:

Terminal

```
pip install "sglang[all]"
python -m sglang.launch_server --model-path unsloth/Llama-3.2-1B-Instruct --port 30000 --enable-metrics
python load_test.py --url http://localhost:30000 --model unsloth/Llama-3.2-1B-Instruct
```

Running vLLM vs SGLang on the same model and table is a strong README. `unverified` SGLang on a T4.

Lightning AI (lightning.ai) gives cloud "Studios" with a VS Code-like IDE and selectable GPUs (T4, L4, A10G…) and a monthly free-credit allowance — see **lightning.ai/pricing** for current numbers. Same commands, but no Colab notebook quirks, and you can use `vllm serve` in a normal terminal.

8 Write the README with your numbers

README_template.md

```
# Serving Llama-3.2-1B-Instruct with vLLM on a free T4

**What:** an OpenAI-compatible LLM endpoint I deployed and load-tested myself.
**How:** vLLM <version> on Google Colab (NVIDIA T4, 15 GB), fp16 (`--dtype half`), max context <N>
tokens.
**Why it matters:** measured latency/throughput trade-off as concurrency grows -> sizing and cost
input.

## Run it
    pip install vllm
    vllm serve unsloth/Llama-3.2-1B-Instruct --dtype half --max-model-len 4096
    python load_test.py --concurrency 1 4 16 --requests 32 --max-tokens 128

## Results (fill in from load_test.py + metrics_summary.py)
| concurrency | TTFT p50 (ms) | TTFT p95 (ms) | ITL p50 (ms) | output tok/s | req/s |
|---|---|---|---|---|---|
| 1  |  |  |  |  |  |
| 4  |  |  |  |  |  |
| 16 |  |  |  |  |  |

Server-side (from /metrics): avg TTFT __ ms, avg ITL __ ms, KV-cache peak __ %.

## What I learned
- Throughput scales with concurrency until ____; after that TTFT climbs because ____.
- Cost: T4 at $__/h -> ~$__ per 1M output tokens at concurrency __.
- Next: try SGLang / a 4-bit AWQ 8B model / an L4 GPU.
```

Done when

- ☐ `curl` to your endpoint returns a completion
- ☐ You read TTFT, inter-token latency and throughput from `/metrics` and from your load test
- ☐ Results table at 1 / 4 / 16 concurrency is in the README, with one sentence on what changes and why
- ☐ You can say what it costs per hour / per million tokens and when you'd choose a bigger GPU or a 4-bit model

Common errors & fixes

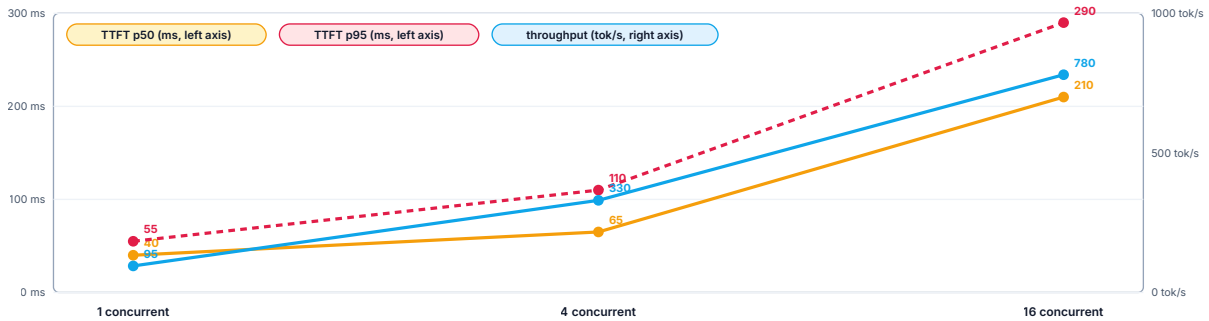
Symptom	Fix
`CUDA out of memory` at start-up	Lower `--max-model-len` (KV cache) and/or `--gpu-memory-utilization 0.8`; use the 1B model.
`Bfloat16 is only supported on GPUs with compute capability ≥ 8.0`	You forgot `--dtype half` (T4 = compute capability 7.5).
Server never becomes healthy	Read `!tail -50 vllm.log`. Often a still-running download, a torch/vLLM version clash after `pip install` (restart the session), or an unsupported-GPU error (see the warning above).
Gated model / 401 from Hugging Face	Meta's own repo is gated; `unsloth/Llama-3.2-1B-Instruct` is an open mirror. Or log in with `huggingface-cli login`.
`Connection refused` from load_test	Wrong port, or the server died — check `vllm.log`; in Colab both must run in the same runtime.
Colab disconnects mid-benchmark	Free tier limits; keep runs short, save results to Drive immediately.

03 · Evals Deploy — evals, worked example

Metric	Fake-server test `measured`	T4 example `illustrative values`	How to read it
TTFT p50 @1 / @8 or @16	53 / 58 ms (@8)	40 / 210 ms (@16)	Time users stare at a blank screen; rises with queueing
TTFT p95	54 / 59 ms	55 / 290 ms	Tail — what the unluckiest 5% feel; SLAs live here
ITL p50	10.2 / 10.3 ms	12 / 25 ms	Streaming smoothness; ~20+ tokens/s per user reads comfortably
Output tok/s (all users)	84 / 655	95 / 780	Capacity; divides into GPU €/h for cost per token
Failed requests	0 / 0	0 / 0	Any non-zero under load is a red flag
KV cache usage	n/a	0.18 → 0.71	Near 1.0 → preemption, latency spikes

The fake-server numbers only prove the measuring scripts work — the server sleeps a fixed 10 ms per token. They say nothing about GPU performance.

Latency vs concurrency



ILLUSTRATIVE VALUES — shows the typical shape (throughput ↑, tail latency ↑ faster). Replace with your measured table.

Cost per 1M output tokens = GPU price per hour ÷ (tok/s × 3,600) × 1,000,000. Example: a GPU at €0.60/h serving 780 tok/s ≈ €0.21 per 1M tokens at full load — and proportionally more when the GPU idles. `illustrative values`

03 · Review

Deploy — reviewer's checklist

Ask the engineer	Artefact to demand	Good answer	Red flag
Which GPU, how much memory, which config?	nvidia-smi output + serve command	Exact GPU, flags with reasons	“Default settings”
TTFT p50 and p95 per concurrency?	load_test table	Both percentiles, 3+ levels	Average only, one level
Inter-token latency?	ITL p50/p95	Stable as load grows (or explained)	Not measured
Throughput at concurrency?	tok/s and req/s per level	Curve with a knee identified	One headline number
Error rate under load?	failed column + server logs	0, or failures explained (timeouts, OOM)	Failures dropped from the stats
Cost per 1K / 1M tokens?	Formula + GPU price + utilisation	Includes idle time assumption	Compared to API list price at 100% util
Model and version pinning?	requirements + model revision	Exact versions	'latest'
Cold vs warm?	Seeds, first-run numbers	Warm-up excluded deliberately, random prompts	Same prompt repeated (prefix cache)

03 · Errors Deploy — error gallery

Messages are representative of what vLLM / the OpenAI-style API return; exact wording varies by version.

`illustrative values`

CUDA out of memory at start-up

```
ValueError: The model's max seq len (131072)
is larger than the maximum number of tokens
that can be stored in KV cache …
# or: torch.OutOfMemoryError: CUDA out of memory
```

Symptom	Server never becomes healthy.
How to spot	vllm.log at start-up; nvidia-smi memory.
Likely causes	Default context length of the model is huge; memory util too high/low; model too big.
Fix	--max-model-len 4096, tune --gpu-memory-utilization, smaller or 4-bit model.

TTFT spike under load

```
concurrency  TTFT p50  TTFT p95
1            40 ms     55 ms
16           210 ms    1,900 ms   ← tail
vllm:num_requests_waiting 9
```

Symptom	Fine in demo, slow with real users.
How to spot	p95 vs p50 gap; num_requests_waiting > 0; queue_time ↑.
Likely causes	Batch slots exhausted, KV cache full, long prompts prefilling.
Fix	Raise max-num-seqs if memory allows, add replicas, cap prompt length, set SLA per tier.

Metrics measured on a warm cache only

```
run 1: TTFT p50 180 ms
run 2: TTFT p50  35 ms   ← reported
# same prompt every request
```

Symptom	Numbers too good to be true.
How to spot	Load test uses one fixed prompt; no --seed change.
Likely causes	Prefix caching reuses work for repeated prompts.
Fix	Random or varied prompts, new seed per run, report cold and warm separately.

bf16 on a T4

```
ValueError: Bfloat16 is only supported on GPUs
with compute capability of at least 8.0.
Your Tesla T4 GPU has compute capability 7.5.
```

Symptom	Server exits immediately.
How to spot	First lines of vllm.log.
Likely causes	Model config defaults to bf16.
Fix	--dtype half (fp16) on T4.

Request longer than the context

```
400 Bad Request: This model's maximum context
length is 4096 tokens. However, you requested
4310 tokens …
```

Symptom	Some requests fail, only with long inputs.
How to spot	Failed column in the load test; 400s in server log.
Likely causes	--max-model-len lowered without capping inputs; RAG stuffing too many chunks.
Fix	Trim context, cap max_tokens, align limits across services.

Throughput counted wrong

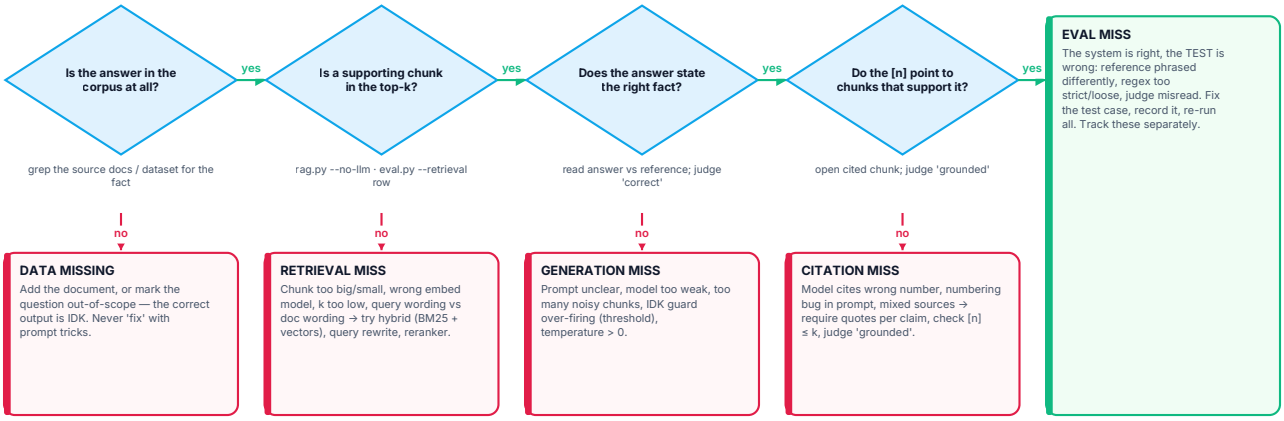
```
client counted SSE chunks = 128
server usage.completion_tokens = 141
```

Symptom	Throughput off by ~10%; vendor comparison unfair.
How to spot	Compare client counts with usage / generation_tokens_total.
Likely causes	Chunks ≠ tokens; usage not requested.
Fix	stream_options.include_usage; use server counters for tokens.

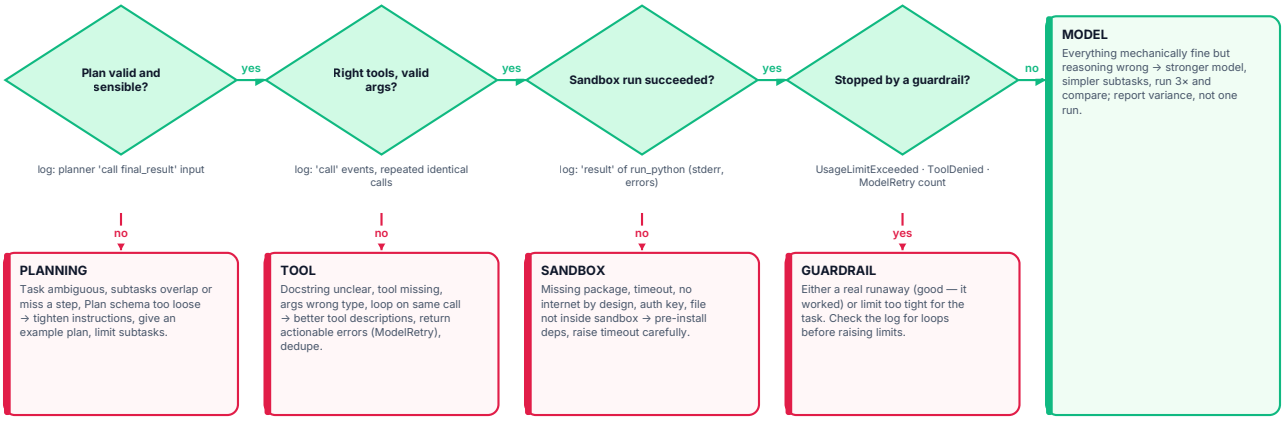
Diagnose Diagnostic decision trees

Start at the left with one failing example, answer each question from the named artefact, and stop at the first red leaf. Fix that class of problem, then re-run the full eval — never just the one example.

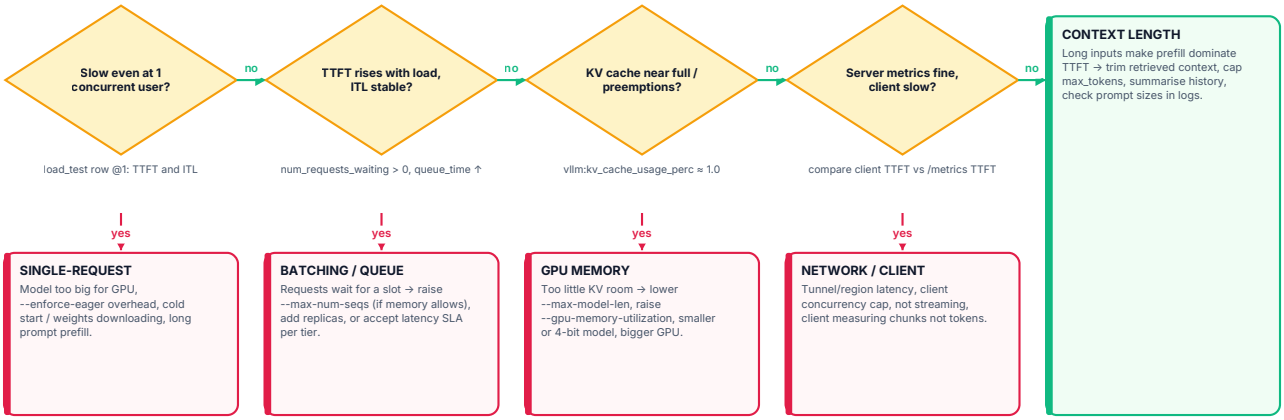
“The answer is wrong” — RAG



“The agent failed” — Agents



“The endpoint is slow” — Deploy



Read How to read an eval report

An annotated example (`illustrative values`). Read it top-down: is the test trustworthy, then retrieval, then answer quality, then cost — and finally where it fails.

```
RAG eval report — v3 vs v2                          (example)
-----------------------------------------------------------
test set      : qa_test_v1.json   n = 100   frozen 2026-10-20
system        : bge-small · k=5 · chunk 400/50 · gpt-4o-mini
runs          : 3 (temperature 0)
                      v2        v3       delta
retrieval hit@5     54%       60%      +6 pp   ③
regex pass          41%       48%      +7 pp
judge correct       58%       63% ±2   +5 pp   ④
judge grounded      52%       61% ±3   +9 pp
IDK on answerable   12%        7%      -5 pp   ⑤
IDK on off-topic    90%       95%      +5 pp
p95 latency       2.9 s     3.1 s              ⑥
cost / 1K queries  €0.42     €0.45
by category        dates 81% · people 74% · lists 22%
```

① Size and provenance: n=100 is the minimum for ±5–10 pp; frozen before tuning?

② Repeated runs: one run hides noise; look for ± spread.

③ Retrieval first: if it's low, generation can't fix it.

④ Judge numbers need a judge–human agreement check.

⑤ Both IDK rates: low on answerable AND high on off-topic.

⑥ Quality gains often cost latency/€ — is it worth it?

⑦ Averages hide clusters: 'lists' at 22% is the real story.

Traps

⚠ Too-small eval set
20 questions → one answer = 5 pp. Differences under ~10 pp are noise.

⚠ Overlap with tuning data
If the same questions chose k and chunk size, the score is optimistic. Ask for a separate dev set.

⚠ LLM-judge bias
Judges favour long, confident answers and their own model family. Use a different model, a rubric, and a human spot-check.

⚠ Averages hiding failure clusters
Break down by question type, document, language. One category at 20% can sink adoption.

⚠ Non-deterministic runs reported once
Ask for ≥3 runs and the spread; temperature 0 doesn't make APIs fully deterministic.

Part 4 Publish: what / how / results

The carousel's last slide: put all three on GitHub, each with a README that says **what** it does, **how** it works, and **results** (numbers). Root README:

README.md (root)

```
# AI projects — Leonard Dumitru
| Project | What | Key result |
|---|---|---|
| [01-rag](01-rag) | Q&A over documents with page-level citations and an "I don't know" guard | retrieval
hit-rate 60% @k=5 → __% after tuning; judge pass-rate __% |
| [02-agents](02-agents) | Planner + parallel sub-agents, sandboxed code, budget caps, human approval | __/
__ tasks pass; avg $__ per task |
| [03-deploy](03-deploy) | Llama-3.2-1B on vLLM, free T4 | TTFT p50 __ ms @1 user → __ ms @16; __ tok/s |

Stack: Python 3.11, sentence-transformers, Chroma/FAISS, OpenRouter, Pydantic AI, E2B, vLLM.
```

> **Hiring managers skim.** Put a 20-second GIF or screenshot of each project at the top of its README (macOS: Cmd+Shift+5; Windows: Win+Shift+S / Xbox Game Bar), and keep the numbers in the first screen.

Appendix

Versions, verification, links

Package	Version checked (30 Oct 2026)
chromadb	1.5.9
sentence-transformers	6.2.0 (torch 2.15.0)
datasets / pypdf / faiss-cpu	5.1.0 / 6.21.4 / 1.15.1
openai (client)	3.41.0
pydantic-ai-slim	2.63.0
e2b-code-interpreter	2.11.6
vllm / sglang	0.32.0 / 0.5.23 (latest on PyPI; not run — no GPU)

Testing status

- **01 RAG:** full ingest of rag-mini-wikipedia (3,210 chunks, ~1 min on CPU), retrieval, the IDK distance guard path, retrieval eval (60/100), FAISS search. *Not run:* LLM answers and LLM-judge (no API key used).
- **02 Agents:** full run with Pydantic AI `TestModel`: planner → parallel worker → writer, all four tools, JSONL logging, approval prompt (y/N), ModelRetry, sandbox error handling. E2B/Docker call signatures checked against the installed SDK. *Not run:* real model or real E2B sandbox (no keys), Docker (not installed on the test machine); `run_tests.py` only exercised with the dummy model.
- **03 Deploy:** `load_test.py` and `metrics_summary.py` against `fake_server.py`. *Not run:* vLLM/SGLang on a T4; metric names taken from vLLM's docs/source.

Links

- OpenRouter keys: openrouter.ai/keys · models: openrouter.ai/models
- E2B docs: e2b.dev/docs · Pydantic AI: ai.pydantic.dev
- Dataset: huggingface.co/datasets/rag-datasets/rag-mini-wikipedia
- vLLM metrics: docs.vllm.ai/en/latest/usage/metrics.html · vLLM benchmarks: docs.vllm.ai (bench serve)
- Courses (from the carousel): DeepLearning.AI “Building and Evaluating Advanced RAG”; Hugging Face Agents Course (huggingface.co/learn/agents-course)
- Lightning AI pricing: lightning.ai/pricing